

Table of Contents

- [Table of Contents](#)
- [Problem 1 — Hackermannn!!!](#)
 - [Complete C Solution](#)
- [Problem 2 — Thesis Marketing](#)
 - [Complete C Solution](#)
- [Problem 3 — Mission: Welcome](#)
 - [Solution \(C\)](#)
- [Problem 4 — K-Distinct Substrings](#)
 - [Solution \(C\)](#)
- [Problem 5 — The Treasure Caravan](#)
 - [Solution \(C\)](#)
- [Problem 6 — The Number Pyramid](#)
 - [Solution \(C\)](#)
- [Problem 7 — Restoring the Oath](#)
 - [Solution \(C\)](#)
- [Problem 8 — Harmonics of the Lexicon](#)
 - [Solution \(C\)](#)
- [Problem 9 — The Guildmaster's Selection](#)
 - [Solution](#)
- [Problem 10 — Paths of Least Peril](#)
 - [Solution](#)
- [Problem 11 — Hackerman's Network Sweep](#)
 - [Solution](#)

Problem 1 — Hackermannn!!!

A curious student from the Department of Computer Science and Engineering (CSE) at Shanto-Mariam University of Creative Technology (SMUCT) is determined to obtain the hidden problem set for the upcoming SMUCT Intra University Programming Contest. To

do so, the student secretly enters the contest coordinator's office and finds a notebook containing **n** possible access codes. Each access code is a non-empty string consisting of lowercase English letters. After examining the contest system, the student discovers that the system stores exactly **one** of the access codes from the notebook as the administrator password. The authentication system considers two access codes **equivalent** according to the following rules:

- Two access codes **A** and **B** are directly equivalent if they share at least one common letter.
- Two access codes **A** and **B** are also considered equivalent if there exists another access code **C** from the notebook such that **A** is equivalent to **C**, and **C** is equivalent to **B**.

If the actual administrator password is an access code, then entering **any equivalent access code** will successfully unlock the contest system. For example, suppose the notebook contains the following access codes:

a
b
ab
d

Then the access codes "a" , "b" , and "ab" are all equivalent because they are connected through common letters, while "d" forms a separate equivalence group.

- If the administrator password is "b" , the student can gain access using "a" , "b" , or "ab" .
- If the administrator password is "d" , only "d" will work.

Since the student does not know which access code is actually being used by the contest system, they want to determine the **minimum number of access codes** they must try to guarantee gaining access to the hidden problem set.

Input Format

The first line contains an integer **n** ($1 \leq n \leq 2 \times 10^5$) The next **n** lines each contain a non-empty string **s_i** consisting of lowercase English letters.

Constraints

- $1 \leq n \leq 2 \times 10^5$
- Each string has length at most **50**
- Some strings may be identical.
- The total length of all strings does not exceed **10^6**

Output Format

Print a single integer — the minimum number of access codes the student must try to guarantee access to the contest system.

Example

Input

```
4
a
b
ab
d
```

Output

2

Explanation

There are two equivalence groups:

Group 1:

a

b

ab

Group 2:

d

Trying one string from each group guarantees success. Therefore, the answer is **2**.

Complete C Solution

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAXN 200005
#define MAXLEN 55

/* ----- Disjoint Set Union ----- */

int parent[MAXN];
int rankSet[MAXN];

int Find(int x)
{
    if (parent[x] == x)
        return x;

    parent[x] = Find(parent[x]);
    return parent[x];
}

void Union(int a, int b)
{
    a = Find(a);
    b = Find(b);

    if (a == b)
        return;

    if (rankSet[a] < rankSet[b])
    {
        int temp = a;
        a = b;
        b = temp;
    }

    parent[b] = a;
```

```

        if (rankSet[a] == rankSet[b])
            rankSet[a]++;
    }

    /* ----- Main ----- */

    int main()
    {
        int n;
        scanf("%d", &n);

        /* Initialize DSU */
        for (int i = 1; i ≤ n; i++)
        {
            parent[i] = i;
            rankSet[i] = 0;
        }

        /* Stores first string index containing each character */
        int firstOccurrence[26];

        for (int i = 0; i < 26; i++)
            firstOccurrence[i] = -1;

        char s[MAXLEN];

        for (int i = 1; i ≤ n; i++)
        {
            scanf("%s", s);

            int visited[26] = {0};

            int len = strlen(s);

            for (int j = 0; j < len; j++)
            {
                int ch = s[j] - 'a';

                /* Ignore duplicate letters in same string */

```

```

        if (visited[ch])
            continue;

        visited[ch] = 1;

        if (firstOccurrence[ch] == -1)
        {
            firstOccurrence[ch] = i;
        }
        else
        {
            Union(i, firstOccurrence[ch]);
        }
    }
}

int answer = 0;

for (int i = 1; i ≤ n; i++)
{
    if (Find(i) == i)
        answer++;
}

printf("%d\n", answer);

return 0;
}

```

Code Walkthrough

Global Arrays

```
parent[MAXN]
```

DSU parent array।

```
rankSet[MAXN]
```

Union by Rank-এর জন্য। এতে tree ছোট থাকে এবং Find() খুব দ্রুত কাজ করে।

```
firstOccurrence[26]
```

প্রত্যেক character প্রথম কোন string-এ পাওয়া গিয়েছিল সেটা সংরক্ষণ করে। উদাহরণ:

`firstOccurrence['a'] = 5` মানে প্রথম 'a' দেখা গিয়েছিল **৫ নম্বর string**-এ।

visited[26]

```
int visited[26] = {0};
```

একই string-এর একই character একাধিকবার process না করার জন্য। যেমন `aaaaabbbb` এখানে `a` একবার union করলেই যথেষ্ট।

Find()

```
Find(x)
```

Returns the representative (root) of the set. এখানে


```
parent[x] = Find(parent[x]);
```

হচ্ছে **Path Compression**।

Union()

```
Union(a,b)
```

দুটি component-কে merge করে। Rank ব্যবহার করায় tree balanced থাকে।

Main Loop

প্রত্যেক string-এর জন্য—

1. Character পড়ো
2. Character আগে এসেছে?
3. না এলে firstOccurrence-এ রাখো।
4. আগে এলে সেই string-এর সাথে Union করো।

Final Answer

সব string-এর root বের করো। যেগুলো নিজেরাই root— $\text{parent}[i] = i$ তাদের সংখ্যা =
Connected Components = Answer।

Explanation

1. What is the problem asking?

প্রথমে Problem-টা খুব ভালোভাবে বুঝতে হবে। আমাদের কাছে n টি string আছে। এর মধ্যে **একটিই** আসল password। কিন্তু কোনটা আসল password সেটা আমরা জানি না। তবে একটা গুরুত্বপূর্ণ তথ্য দেওয়া আছে— যদি দুইটা string **Equivalent** হয়, তাহলে একটার বদলে আরেকটা ব্যবহার করলেও password accept হবে। অর্থাৎ, আমাদের আসল লক্ষ্য password বের করা না। আমাদের লক্ষ্য হলো—

Minimum কয়টি string try করলে যেকোনো password হলেও নিশ্চিতভাবে login করা যাবে?

2. Thinking Process (Bootcamp Style)

এই অংশটা খুব গুরুত্বপূর্ণ। Contest-এ বসে solution মুখস্থ করে লাভ নেই। Solution কীভাবে মাথায় আসে, সেটাই শিখতে হবে। চলো Step by Step ভাবি।

Step 1

ধরো Input হলো—

4

a

b

ab

d

প্রথম প্রশ্ন— যদি password হয় a তাহলে কি শুধু a দিয়েই login হবে? না। Problem বলছে— Equivalent password-ও accept হবে।

Step 2

তাহলে ab দিয়েও login হবে? হ্যাঁ। কারণ a এবং ab এর মধ্যে common character a আছে। তাই Equivalent।

Step 3

এখন b কি Equivalent?

ab

↓

b

Common character b আছে। তাই Equivalent।

Step 4

এখন আসল প্রশ্ন। a এবং b এর মধ্যে তো common character নেই। তাহলে কি ওরা আলাদা? প্রথমে তাই মনে হবে। কিন্তু Problem-টা আবার পড়ো। বলছে— Indirect equivalence-ও valid. অর্থাৎ

a

↓

ab

↓

b

এই path থাকলেই $a \equiv b$

Step 5

এখন আরেকটা string দেখি। d এর সাথে

a

b

ab

কোন common character নেই। এবং indirect connection-ও নেই। তাই d নিজের আলাদা group।

Step 6

এখন পুরো Input-টা group করলে পাই—

Group 1

a

b

ab

Group 2

d

Step 7

এখন যদি Password থাকে— Group 1 তাহলে Group 1 থেকে যেকোনো একটা string দিলেই হবে।
আবার যদি Password থাকে— Group 2 তাহলে Group 2 থেকে একটা দিতে হবে। অর্থাৎ—

প্রতিটি Group থেকে
কমপক্ষে একটি string
try করতেই হবে।

Step 8

এখন Question পুরো পাল্টে গেল। আগে মনে হচ্ছিল— Password Problem এখন বুঝলাম—
Group Counting Problem

3. Hidden Observation

Contest-এর সবচেয়ে গুরুত্বপূর্ণ Observation এটা। Problem-এ কোথাও Graph শব্দটা নেই। কোথাও Connected Component শব্দটাও নেই। তবুও Problem আসলে Graph। কেন? কারণ— দুইটা string-এর মধ্যে relation আছে। Relation মানেই— Graph বানানো যায়।

Node কী? একটি String

Edge কী? যদি কমপক্ষে একটি common character থাকে।

তাহলে পুরো Problem দাঁড়ালো—

একটি Graph-এর

Connected Component

গুনতে হবে।

4. Brute Force কেন সম্ভব না?

প্রথম Idea কী আসবে? সব string-এর সাথে সব string compare করি। অর্থাৎ—

```
for(i)
```

```
    for(j)
```

Complexity $O(n^2)$ Problem-এ $n = 200000$ অর্থাৎ

200000×200000

=

$40,000,000,000$

প্রায় ৪০ বিলিয়ন Comparison। Contest-এ এটা অসম্ভব। TLE নিশ্চিত।

5. এবার Observation বদলাই

আমরা কী compare করছি? Strings Strings অনেক। কিন্তু Character? মাত্র ২৬ টা। এটাই পুরো Problem-এর Hidden Trick। String অনেক। Character খুব কম। তাই String-কে String-এর সাথে compare না করে— Character-কে bridge বানানো যায়।

উদাহরণ—

String 1

abc

String 9

az

দুইটার মধ্যে a Common। তাহলে

String 1

এবং

String 9

একই Group।

6. এখানেই DSU মাথায় আসে

যখনই দেখবে—

Merge

Groups

Connected

Equivalent

Union

তখন প্রথম Data Structure যেটা মাথায় আসা উচিত— Disjoint Set Union অথবা Union Find

DSU কী করবে? প্রথমে—

সব String

নিজের নিজের Group।

তারপর— যখনই Common Character পাওয়া যাবে— Union() করে দিবে। শেষে—

কয়টা Root আছে?

=

কয়টা Group আছে?

=

Answer

Bootcamp Takeaway

Contest-এ এই Problem-এর সবচেয়ে বড় challenge ছিল **DSU implement করা নয়**। আসল challenge ছিল—

"Strings compare করব" এই চিন্তা থেকে বের হয়ে **"Characters দিয়ে Strings connect করব"** — এই Observation-এ পৌঁছানো।

Competitive Programming-এ অনেক সমস্যার সমাধান algorithm জানার চেয়ে **সঠিক observation** করার উপর বেশি নির্ভর করে। এই problem তার একটি দারুণ উদাহরণ।

7. Building the Algorithm

এখন পর্যন্ত আমরা বুঝেছি—

- প্রতিটি **String = একটি Node**
- Common Character থাকলে তারা Connected
- Indirect Connection-ও Valid
- Answer = Connected Components

এখন প্রশ্ন— **Algorithm কীভাবে বানাবো?**

Step 1 : শুরুতে সবাই আলাদা

ধরো,

a
b
ab
d

শুরুতে প্রত্যেকটা String নিজের Group-এ থাকবে।

String 1 : {1}

String 2 : {2}

String 3 : {3}

String 4 : {4}

DSU Initialization

parent[1] = 1
parent[2] = 2
parent[3] = 3
parent[4] = 4

Step 2 : Character-এর First Occurrence মনে রাখি

আমরা একটা Array রাখব—

```
int firstOccurrence[26];
```

শুরুতে

```
a → -1  
b → -1  
c → -1  
...  
z → -1
```

মানে— এখনও কোনো Character দেখা যায়নি।

Step 3 : প্রথম String

String 1

a

এখানে 'a' প্রথমবার এসেছে। তাই

```
firstOccurrence['a']  
=
```

String 1

Array

$a \rightarrow 1$

$b \rightarrow -1$

$c \rightarrow -1$

...

কোনো Union লাগবে না।

Step 4 : দ্বিতীয় String

String 2

b

প্রথমবার 'b' দেখা গেল। $b \rightarrow 2$ এখনও কোনো Union লাগবে না।

Step 5 : তৃতীয় String

ab এখন Character ধরে দেখি। প্রথম Character a আগেই দেখা হয়েছে।

firstOccurrence[a]

=

1

মানে

String 3

এবং

String 1

Connected। তাই Union(3,1)

এরপর b আগেই দেখা হয়েছে।

firstOccurrence[b]

=

2

তাই Union(3,2) এখন কী হলো? আগে ছিল—

1

2

3

তিনটা আলাদা Group। এখন

Union(3,1)

Union(3,2)

এর পরে—

$\{1, 2, 3\}$

একটাই Group। DSU নিজে থেকেই Merge করে ফেলেছে।

Step 6 : চতুর্থ String

d প্রথমবার এসেছে। তাই $d \rightarrow 4$ Union লাগবে না।

Step 7 : সব String শেষ

এখন DSU-এর ভিতরে Group থাকবে—

$\{1, 2, 3\}$

$\{4\}$

অর্থাৎ Connected Components = 2 Answer 2

8. Duplicate Character কেন Ignore করি?

ধরো aaaaabbbbb এখন যদি প্রতিটি a এর জন্য Union() করো, তাহলে

Union()

Union()

Union()

Union()

অপ্রয়োজনীয় কাজ হবে। একই তথ্য বারবার Process করা হবে। তাই

```
int visited[26];
```

ব্যবহার করি। প্রথমবার a Process করব। পরের a গুলো Skip।

Example

String

aaabbbccc

Actual Processing

a

b

c

মাত্র তিনবার।

9. পুরো Algorithm এক নজরে

Initialize DSU

↓

Initialize firstOccurrence[] = -1

↓

প্রতিটি String পড়ো

↓

প্রতিটি Unique Character-এর জন্য

↓

Character আগে দেখা যায়নি?

|

|— হ্যাঁ

| firstOccurrence[c]=current

|

|— না

Union(current,
 firstOccurrence[c])

↓

সব String শেষ

↓

DSU-এর Root Count করো

↓

Print Answer

10. Algorithm Proof (Why Does It Work?)

এটা Contest Editorial-এর সবচেয়ে গুরুত্বপূর্ণ অংশ। আমাদের Algorithm কখনো ভুল Answer দিবে না। কেন? ধরি, দুইটা String-এর Common Character আছে। তাহলে Union() হবে। অর্থাৎ তারা একই Component-এ চলে যাবে।

যদি দুইটা String-এর Direct Connection না থাকে, কিন্তু

A

↓

B

↓

C

এইভাবে Connected থাকে, তাহলেও—

Union(A,B)

Union(B,C)

হওয়ার পরে DSU-এর Property অনুযায়ী

Find(A)

=

Find(C)

হয়ে যাবে। অর্থাৎ Indirect Connection-ও ঠিকমতো Handle হচ্ছে।

আবার, যদি কোনো দুইটা String-এর মধ্যে কোনোভাবেই Connection না থাকে, তাহলে কখনোই Union() হবে না। ফলে তারা দুইটা আলাদা Root-এই থাকবে। অর্থাৎ প্রত্যেকটা Connected Component-এর জন্য একটি আলাদা Root থাকবে। Root-এর সংখ্যা = Connected Component-এর সংখ্যা = Minimum Password Try করার সংখ্যা। অতএব Algorithm সর্বদা Correct।

11. Dry Run (Step by Step)

চলো এবার পুরো Algorithm-টা হাতে-কলমে চালাই। Input

4

a

b

ab

d

Initial State

DSU

```
parent[1] = 1
parent[2] = 2
parent[3] = 3
parent[4] = 4
```

Graphically

1

2

3

4

সবাই নিজের Parent।

firstOccurrence

```
a → -1
b → -1
c → -1
...
z → -1
```

Processing String 1

a Character a আগে দেখা যায়নি। first0ccurrence[a] = 1 DSU

1

2

3

4

কোনো পরিবর্তন নেই।

Processing String 2

b Character b প্রথমবার এসেছে। first0ccurrence[b] = 2 DSU

1

2

3

4

এখনও সবাই আলাদা।

Processing String 3

ab প্রথম Character a আগেই String 1 এ ছিল। তাই Union(3,1) ধরি 1 Root হলো। তাহলে
parent[3] = 1 এখন

1
|
3

2

4

এরপর b আগেই String 2 এ ছিল। তাই Union(3,2) এখন

Find(3)

=

1

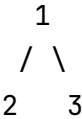
এবং

Find(2)

=

2

Union করলে parent[2]=1 এখন DSU



4

দেখো, আমরা কখনো Union(1,2) করিনি। তবুও

1

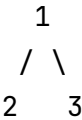
2

3

একই Group হয়ে গেছে। এটাই DSU-এর আসল শক্তি।

Processing String 4

d প্রথমবার এসেছে। firstOccurrence[d]=4 DSU



4

Final Parent Array

Find() চালানোর পরে

Index

1 → 1

2 → 1

3 → 1

4 → 4

Unique Root

1

4

মোট 2 Answer 2

12. Complexity Analysis

ধরি N = Number of Strings এবং L = Total Characters Problem-এ $L \leq 10^6$

Time Complexity

প্রতিটি Character একবার Process হবে। প্রতিবার

Find()

Union()

চলবে। DSU with

- Path Compression
- Union by Rank

এর Complexity $O(\alpha(N))$ এখানে $\alpha(N)$ মানে Inverse Ackermann Function। বাস্তবে $\alpha(N) \leq 5$ অর্থাৎ প্রায় Constant। তাই পুরো Complexity $O(L \times \alpha(N))$ Practical Contest Complexity

$\approx O(L)$

$\approx O(10^6)$

যা সহজেই Accept হবে।

Space Complexity

DSU Parent $O(N)$ Rank $O(N)$ firstOccurrence $O(26)$ visited $O(26)$ Total $O(N)$

13. Common Mistakes

✗ Mistake 1

সব String Pair Compare করা।

```
for(i)
```

```
    for(j)
```

Complexity $O(N^2)$ TLE।

✗ Mistake 2

একই Character বারবার Process করা। যেমন aaaaaaaa এখানে Union() একবার করলেই যথেষ্ট।

✗ Mistake 3

Duplicate String দেখে Confuse হওয়া।

```
abc
```

```
abc
```

এরা দুইটা আলাদা String। দুইটা আলাদা Node। কিন্তু অবশ্যই একই Component।

✗ Mistake 4

শেষে `parent[i]=i` গুনে ফেলা। বরং `Find(i)` ডেকে Root Update করে তারপর Count করতে হবে। সঠিক Code

```
if (Find(i) == i)
    answer++;
```

14. Pattern Recognition

Contest-এ নিচের শব্দগুলো দেখলেই DSU মাথায় আসা উচিত—

- Connected
- Equivalent
- Merge
- Union
- Groups
- Components
- Same Set
- Friend
- Network
- Similar
- Relation
- Connected Through
- Indirectly Connected

এই Problem-এ Equivalent এবং Indirectly Equivalent এই দুইটা Keyword-ই পুরো Solution-এর Hint।

15. Similar Problems

এই Pattern আরও অনেক Contest-এ এসেছে।

Codeforces

- Secret Passwords (DSU)
- Learning Languages
- People Are Leaving

CSES

- Road Construction
- Road Reparation (DSU Part)

AtCoder

- Union Find Problems

16. Final Bootcamp Summary

এই Problem-এর সবচেয়ে গুরুত্বপূর্ণ শিক্ষা হলো— প্রথমে Problem দেখে মনে হতে পারে এটি **String Problem**। কিন্তু একটু গভীরভাবে ভাবলে বোঝা যায় এটি আসলে **Graph Problem**। আর Graph তৈরি করতে গেলেও String-to-String Compare করার দরকার নেই। মাত্র **২৬টি Character**-কেই Bridge হিসেবে ব্যবহার করে সমস্ত Connected Component বের করা যায়। Contest-এ অনেক সময় Algorithm জানা যথেষ্ট নয়। যে Observation তোমাকে Brute Force থেকে Efficient Solution-এর দিকে নিয়ে যায়, সেটাই একজন ভালো Competitive Programmer-এর আসল দক্ষতা।

Bootcamp Checklist

Problem Type Graph + DSU Core Data Structure Disjoint Set Union Core Observation

Character acts as a bridge
between strings.

Time Complexity

$O(\text{Total Characters} \times \alpha(N))$
 $\approx O(10^6)$

Space Complexity $O(N)$ Difficulty

★★★★☆

(Observation-based DSU)

Contest Tags

DSU

Union Find

Graph

Connected Components

Strings

Observation

Problem 2 — Thesis Marketing

The Department of Computer Science and Engineering (CSE) is assigning undergraduate students to thesis advisors. There are n students, numbered from **1** to n . Each student either:

- has **no thesis advisor** (they are the principal researcher leading an independent project), or
- has **exactly one direct thesis advisor**, who is another researcher with a different ID.

A researcher **A** is considered the academic advisor of another researcher **B** if at least one of the following is true:

- Researcher **A** is the direct thesis advisor of researcher **B**.
- Researcher **B** has a direct thesis advisor **C**, and researcher **A** is the academic advisor of **C**.

It is guaranteed that the advising structure contains **no cycles**. In other words, no researcher can directly or indirectly become the advisor of their own advisor. At the end of the semester, the department is organizing a **Research Progress Review**. All researchers must be divided into the **minimum number of presentation sessions**, where each researcher is assigned to exactly one session. To avoid conflicts of interest during evaluation,

No presentation session may contain both a thesis advisor and any of their direct or indirect advisees.

Determine the **minimum number of presentation sessions required**.

Input Format

The first line contains an integer n — the number of researchers. The next n lines each contain an integer p_i .

- If $p_i = -1$, researcher i has no thesis advisor.
- Otherwise, p_i is the ID of the direct thesis advisor of researcher i .

It is guaranteed that:

- No researcher is their own direct thesis advisor.
- The advising hierarchy contains no cycles.

Constraints

- $1 \leq n \leq 2000$
- $1 \leq p_i \leq n$ or $p_i = -1$

Output Format

Print a single integer — the **minimum number of presentation sessions required**.

Example 1

Input

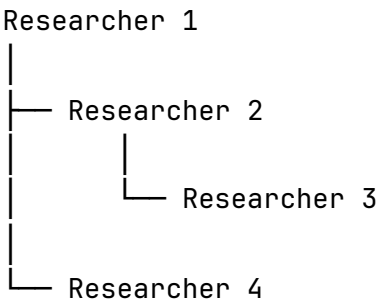
```
5
-1
1
2
1
-1
```

Output

3

Explanation

The advising hierarchy is:



Researcher 5

One possible assignment is

Session 1:
Researcher 1
Researcher 5

Session 2:
Researcher 2
Researcher 4

Session 3:
Researcher 3

No advisor appears in the same session as any of their direct or indirect advisees.
Therefore, the minimum number of sessions is

3

Complete C Solution

```
#include <stdio.h>

#define MAXN 2005

int parent[MAXN];
int depth[MAXN];

/* Returns the depth (level) of a node from the root.
   Root depth = 1
*/
int getDepth(int node)
{
    if (node == -1)
        return 0;

    if (depth[node] != 0)
        return depth[node];

    depth[node] = getDepth(parent[node]) + 1;

    return depth[node];
}

int main()
{
    int n;
    scanf("%d", &n);

    for (int i = 1; i ≤ n; i++)
        scanf("%d", &parent[i]);

    int answer = 0;

    for (int i = 1; i ≤ n; i++)
    {
        int currentDepth = getDepth(i);
```

```
        if (currentDepth > answer)
            answer = currentDepth;
    }

    printf("%d\n", answer);

    return 0;
}
```

Thinking Process

1. What is the problem asking?

প্রথমে Problem-টা দেখে অনেকেই ভাববে—

"Students-দের Session-এ ভাগ করতে হবে।"

কিন্তু CP-তে কখনোই Problem-এর Story দেখে Solution বের করা যায় না। Story বাদ দাও। Condition গুলো পড়ো। Problem আমাদের বলছে—

- একজন Student-এর একজন Parent (Advisor) থাকতে পারে।
- Parent-এরও Parent থাকতে পারে।
- কোনো Cycle নেই।
- Advisor এবং তার কোনো Descendant একই Session-এ থাকতে পারবে না।
- Minimum সংখ্যক Session বের করতে হবে।

এখনও Graph বা Tree কোথাও লেখা নেই। কিন্তু CP-তে Story-এর নিচে আসল Structure লুকিয়ে থাকে।

2. First Observation

প্রতিটি Student-এর কয়জন Parent? সর্বোচ্চ ১ জন। একজন Student-এর Child কয়জন হতে পারে? যত খুশি। এটা কোন Structure? Tree যদি একাধিক Root থাকে? Forest অর্থাৎ— পুরো Problem আসলে একটা Forest Problem

3. Draw the Structure

ধরো Input হলো—

7

-1

1

2

1

-1

5

6

Tree আঁকলে—

Tree 1

```

      1
     / \
    2   4
    |
    3
  
```

আরেকটা Tree

Tree 2

5
|
6
|
7

এখন Problem অনেক সহজ দেখাচ্ছে।

4. এখন Session নিয়ে ভাবি

ধরি

1
|
2
|
3

এই Chain আছে। Question— একই Session-এ

1

2

রাখা যাবে? না। কারণ 1 Advisor। 2 Advisee।

আচ্ছা,

2

3

? এটাও না।

1

3

? এটাও না। কারণ Problem বলেছে— Direct অথবা Indirect Advisee দুজন একই Session-এ থাকতে পারবে না।

5. তাহলে Session কীভাবে হবে?

Chain

1

↓

2

↓

3

Session

Session 1

1

Session 2

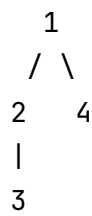
2

Session 3

3

Minimum Session 3

6. এবার একটু বড় Example



এখন Session বানাই।

Session 1

1

Session 2

2

4

কারণ

2

এবং

4

একজন আরেকজনের Advisor না। তাই একসাথে থাকতে পারে।

Session 3

3

Total 3 Session।

7. এখানেই Hidden Observation

Tree-এর Height কত?

1

↓

2

↓

3

Height 3 Answer? 3 কাকতালীয়? চলো আরেকটা দেখি।

5

↓

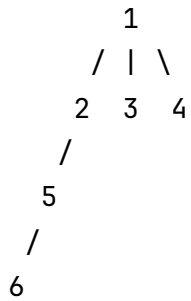
6

↓

7

Height 3 Session? 3 আবার Match।

8. এবার আরও বড় Tree



Height 4 Session? চেষ্টা করি।

Session 1

1

Session 2

2

3

4

Session 3

5

Session 4

6

মোট 4 আবার একই।

9. তাহলে Answer কী?

এখন Observation দাঁড়ালো—

Answer

=

Tree-এর Height

যদি অনেকগুলো Tree থাকে? Forest তাহলে? সব Tree-এর Height বের করো। Maximum Height ই হবে Answer। কারণ সব Session সবার জন্য Common। এক Tree-এর Height যদি 5 হয়, আরেকটার 2 হয়, তাহলে কমপক্ষে 5 Session লাগবেই।

10. কেন Maximum Height?

ধরি Tree A Height 5 Tree B Height 2 তুমি যদি 4 Session বানাও, Tree A-এর একটা Chain-এর দুইজন একই Session-এ পড়ে যাবে। Rule ভেঙে যাবে। অর্থাৎ

Answer

$\geq$ Maximum Height

আবার Maximum Height সংখ্যক Session দিয়েই Level অনুযায়ী সবাইকে বসানো সম্ভব। অর্থাৎ

Answer

$\leq$ Maximum Height

দুইটাই সত্য। তাই

Answer

=

Maximum Height

এটাই এই Problem-এর সবচেয়ে গুরুত্বপূর্ণ Mathematical Observation।

Building the Algorithm

এখন পর্যন্ত আমরা বুঝেছি—

- পুরো Structure একটি **Forest (Collection of Trees)**।
- Answer হলো **Maximum Depth**।
- এখন প্রশ্ন—

Maximum Depth কীভাবে বের করব?

চলো Step by Step Algorithm তৈরি করি।

Step 1 : Parent Array পড়ি

Input

7

-1

1

2

1

-1

5

6

এখানে

`parent[1] = -1`

`parent[2] = 1`

`parent[3] = 2`

`parent[4] = 1`

`parent[5] = -1`

`parent[6] = 5`

`parent[7] = 6`

আমাদের কাছে পুরো Tree Ready আছে।

Step 2 : একটি Node-এর Depth কী?

ধরি

1
|
2
|
3

Depth সংজ্ঞা— Root-এর Depth = 1 তাহলে

$$\text{Depth}(1) = 1$$

$$\text{Depth}(2) = 2$$

$$\text{Depth}(3) = 3$$

অর্থাৎ

$$\text{Depth}(\text{Node})$$

=

$$\text{Depth}(\text{Parent})$$

+

1

এটাই পুরো Problem-এর Formula।

Step 3 : Root-এর Depth কত?

Root-এর Parent -1 অর্থাৎ

```
getDepth(-1)
```

=

0

তাই

Root Depth

=

0 + 1

=

1

এজন্য Code-এ লিখেছি—

```
if(node == -1)
    return 0;
```

Step 4 : Recursive Formula

ধরি 6 নম্বর Node-এর Depth বের করতে হবে। তার Parent 5 অর্থাৎ

Depth(6)

=

Depth(5)

+

1

আবার

Depth(5)

=

Depth(-1)

+

1

তাই

Depth(6)

=

2

একইভাবে

Depth(7)

=

Depth(6)

+

1

=

3

Step 5 : Recursive Function

তাই Function হবে—

```
getDepth(node)
```

যদি node == -1 হয় Return 0 অন্যথায়

Depth(node)

=

Depth(parent[node])

+

1

এটাই সম্পূর্ণ Logic।

Step 6 : একই Depth বারবার বের করব?

ধরি

```
1
|
2
|
3
|
4
```

প্রথমে Depth(4) বের করলাম। তখন

Depth(3)

Depth(2)

Depth(1)

সবই বের হয়ে গেল। এখন যদি আবার Depth(3) চাই, আবার Root পর্যন্ত যাওয়ার দরকার আছে? না। আগেই বের করা হয়েছে।

Step 7 : Memoization

তাই একটা Array রাখব।

depth[MAXN]

যদি

depth[node]

$\neq$

0

তাহলে সরাসরি Return।

```
if(depth[node]  $\neq$  0)
    return depth[node];
```

এতে একই Node-এর Depth একবারই Calculate হবে।

Step 8 : সব Node-এর জন্য Depth বের করি

এখন

```
for(i=1;i $\leq$ n;i++)
```

চালাব। প্রত্যেক Node-এর জন্য getDepth(i) Call করব।

উদাহরণ

Depth(1)=1

Depth(2)=2

Depth(3)=3

Depth(4)=2

Depth(5)=1

Depth(6)=2

Depth(7)=3

Step 9 : Maximum Depth রাখি

ধরি `answer = 0` প্রত্যেক Node-এর পরে

```
if(currentDepth > answer)
```

```
    answer = currentDepth;
```

শেষে `answer` হবে Forest-এর Maximum Depth। এটাই Minimum Session।

Step 10 : Visual Summary

Read Input



Store Parent



For Every Node



Find Depth Recursively



Store Depth (Memoization)



Update Maximum Depth



Print Answer

Why This Algorithm Works

ধরি, Tree-এর সবচেয়ে বড় Chain-এর দৈর্ঘ্য k সেই Chain-এর প্রতিটি Node একজন আরেকজনের Direct অথবা Indirect Advisor। অর্থাৎ ওদের প্রত্যেককে আলাদা Session-এ বসাতেই হবে। তাই কমপক্ষে k টি Session লাগবে। আবার, যদি আমরা একই Depth-এর সব Node-কে একই Session-এ রাখি, তাহলে কোনো Advisor এবং তার Descendant একই Session-এ পড়বে না। অর্থাৎ ঠিক k টি Session দিয়েই কাজ হয়ে যায়। সুতরাং,

Minimum Sessions

=

Maximum Depth

Algorithm সর্বদা Correct।

Dry Run

Let's dry run the algorithm using the sample input.

Sample Input

7

-1

1

2

1

-1

5

6

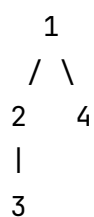
Step 1 : Store the Parent Array

Student : 1 2 3 4 5 6 7

Parent : -1 1 2 1 -1 5 6

From this, we get the following forest.

Tree 1



Tree 2



Step 2 : Initialize the Arrays

Initially,

`depth[] = {0,0,0,0,0,0,0}`

and `answer = 0` A depth of **0** means the node has not been calculated yet.

Step 3 : i = 1

Call

```
getDepth(1)
```

Since `parent[1] = -1` the recursion becomes

```
getDepth(1)
```

↓

```
getDepth(-1)
```

↓

```
0
```

Therefore,

```
Depth(1)
```

```
=
```

```
0 + 1
```

```
=
```

```
1
```

Store $\text{depth}[1] = 1$ Update $\text{answer} = 1$ Current table

Node	Depth
1	1

Step 4 : $i = 2$

Call

```
getDepth(2)
```

Recursion

2

↓

1

↓

Root

We already know $\text{Depth}(1) = 1$ So,

Depth(2)

=

1 + 1

=

2

Store depth[2] = 2 Update answer = 2 Current table

Node	Depth
1	1
2	2

Step 5 : i = 3

Call

getDepth(3)

Recursion

3

↓

2

↓

1

↓

Root

But Depth(2) has already been calculated. Therefore,

Depth(3)

=

2 + 1

=

3

Store depth[3] = 3 Update answer = 3 Current table

Node	Depth
1	1
2	2
3	3

Step 6 : i = 4

Call

```
getDepth(4)
```

Path

4

↓

1

Since $\text{Depth}(1)=1$ we get $\text{Depth}(4)=2$ Store $\text{depth}[4]=2$ Maximum depth is still 3

Step 7 : i = 5

Call

```
getDepth(5)
```

Since it is a root, $\text{Depth}(5)=1$ Store $\text{depth}[5]=1$ No change in the answer.

Step 8 : i = 6

Call

`getDepth(6)`

Path

6

↓

5

↓

Root

Therefore, $\text{Depth}(6)=2$ Store $\text{depth}[6]=2$ Maximum depth remains 3

Step 9 : i = 7

Call

`getDepth(7)`

Path

7

↓

6

↓

5

↓

Root

Since $\text{Depth}(6)=2$ we get $\text{Depth}(7)=3$ Store $\text{depth}[7]=3$ Maximum depth is still 3

Step 10 : Final Depth Table

Student	Depth
1	1
2	2
3	3
4	2
5	1
6	2
7	3

The maximum value in the table is 3 Therefore, Minimum Number of Sessions = 3 which matches the sample output.

Memoization in Action

Notice what happens while calculating `Depth(3)`. The recursion first computes `Depth(1)` and `Depth(2)`. Once they are stored, `Depth(4)` does **not** recompute `Depth(1)`. Similarly, `Depth(7)` uses the already stored value of `Depth(6)` instead of traversing the entire path again. This is exactly why the algorithm runs efficiently.

Complexity Analysis

Whenever we solve a Competitive Programming problem, we should always analyze two things:

1. **Time Complexity**
2. **Space Complexity**

Let's analyze both.

Time Complexity

At first glance, the recursive function may seem expensive because it keeps calling itself. Let's see what actually happens. Assume the tree is

1
|
2
|
3
|
4
|
5

When we call

`getDepth(5)`

the recursion goes like this:

getDepth(5)

↓

getDepth(4)

↓

getDepth(3)

↓

getDepth(2)

↓

getDepth(1)

↓

Root

While returning, we store

depth[1] = 1

depth[2] = 2

depth[3] = 3

depth[4] = 4

depth[5] = 5

Now suppose later we call


```
getDepth(4)
```

Instead of repeating the recursion, the function immediately returns

```
depth[4]
```

because it has already been calculated. The same happens for every other node.

How Many Times Is Each Node Processed?

Every node has only two possibilities.

Case 1

The node's depth has never been calculated. Compute it once.

Case 2

The node's depth is already stored. Return immediately. So, each node is fully processed only **one time**.

Suppose there are N students. Then,

Node 1 → processed once

Node 2 → processed once

...

Node N → processed once

Therefore, the total work is proportional to N . Hence, Time Complexity = $O(N)$

Why Isn't It $O(N^2)$?

Many beginners think,

"We are calling `getDepth()` for every node, so shouldn't it be $O(N^2)$?"

Without memoization, that would indeed be true. For example,

```
1
|
2
|
3
|
4
```

If we repeatedly walked back to the root for every node, the total operations would become

$$1 + 2 + 3 + 4 + \dots$$

$$\approx N^2$$

However, because we store every computed depth, each node is calculated only once. So, the repeated work disappears. This reduces the complexity from

$$O(N^2)$$

↓

$$O(N)$$

Space Complexity

We use two arrays.

```
parent[MAXN]
```

stores the parent of every node. Size N

Another array is

```
depth[MAXN]
```

which stores the calculated depth. Size N

The recursive function also uses the call stack. In the worst case, the tree may look like

```

1
|
2
|
3
|
4
|
...
|
N

```

The recursion depth becomes N . Therefore, the recursion stack also requires $O(N)$ memory.

Total memory usage is therefore

Parent Array

+

Depth Array

+

Recursion Stack

which is still $O(N)$

Final Complexity

Analysis	Complexity
Time Complexity	$O(N)$
Space Complexity	$O(N)$

This is optimal because every student must be examined at least once, making $O(N)$ the best possible time complexity for this problem.

Common Mistakes

Even after understanding the idea, beginners often make a few common mistakes while solving this problem. Let's discuss each of them.

Mistake 1 : Treating It as a Graph Traversal Problem

Many beginners immediately think,

"This is a tree, so I should build an adjacency list and run DFS."

Although that solution works, it is unnecessarily complicated. In this problem, every student already knows their parent. So, there is no need to build the tree explicitly. The **parent array itself contains all the required information**. Using it directly leads to a much simpler solution.

Mistake 2 : Forgetting That There Can Be

Multiple Roots

Many programmers assume there is only one root. For example,

5

-1

1

-1

3

4

This input actually represents two different trees.

Tree 1

1

|

2

Tree 2

3

|

4

|

5

If your solution assumes only one root, it will ignore some students and produce the wrong answer. Always remember:

The input represents a Forest,
not necessarily a single Tree.

Mistake 3 : Using Only One Root to Calculate the Answer

Some beginners first find a root, then calculate only that tree's height. Example

Tree A → Height = 2

Tree B → Height = 5

Correct Answer 5 Wrong Approach Answer = 2 The answer must be the **maximum depth among all trees**.

Mistake 4 : Forgetting Memoization

Suppose the tree is

```
1
|
2
|
3
|
4
|
5
```

Without memoization, calculating `Depth(5)` will visit every ancestor. Later, calculating `Depth(4)` will visit them all again. Then, `Depth(3)` does the same. The same work is repeated many times. This makes the algorithm much slower. Always store the calculated depth.

```
if(depth[node] != 0)
    return depth[node];
```

This single condition changes the solution from

$O(N^2)$

to

$O(N)$

Mistake 5 : Returning the Wrong Base Case

Some beginners write

```
if(node == -1)
    return 1;
```

This is incorrect. The imaginary parent of a root has depth 0 not 1. Otherwise, every depth becomes one greater than its actual value. Correct base case

```
if(node == -1)
    return 0;
```


Mistake 6 : Forgetting to Update the Maximum Answer

Some solutions correctly compute every depth, but never update the maximum value.

Wrong

```
getDepth(i);
```

Correct

```
currentDepth = getDepth(i);
```

```
if(currentDepth > answer)
    answer = currentDepth;
```

Without updating the maximum, the final answer will always be incorrect.

Mistake 7 : Confusing Depth with Number of Children

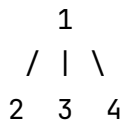
Some beginners think

More Children

=

Greater Depth

This is completely false. Example



Node 1 has three children, but the tree height is only 2. Depth depends on the **longest path from the root**, not on the number of children.

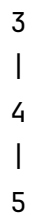
Mistake 8 : Thinking the Answer Is the Number of Trees

Example

Tree 1



Tree 2



There are 2 trees, but the answer is 3 because the tallest tree has depth 3. The number of connected components has nothing to do with the answer.

Quick Checklist Before Submission

Before submitting your solution, make sure that:

- ☒ You considered the input as a **forest**.
- ☒ You used the **parent array** correctly.
- ☒ Your base case returns **0** for **-1**.
- ☒ Every calculated depth is stored (memoization).
- ☒ You calculate the depth for **every** student.
- ☒ You update the maximum depth after every calculation.
- ☒ Your final answer is the **maximum depth**, not the number of trees.

If all of these are correct, your solution should pass all test cases.

Pattern Recognition

One of the most important skills in Competitive Programming is recognizing patterns. The faster you recognize the pattern, the faster you can solve the problem. Let's see what pattern this problem teaches us.

Pattern 1 : Parent Array Represents a Tree

Whenever the input looks like

```
parent[1] = -1
```

```
parent[2] = 1
```

```
parent[3] = 2
```

```
parent[4] = 1
```

immediately think

"This is a Tree (or Forest)."

You do **not** need to build an adjacency list unless the problem specifically requires traversing children. Many problems can be solved directly using the parent array.

Pattern 2 : Parent Depends on Parent

Notice the formula

`Depth(Node)`

=

`Depth(Parent)`

+

1

Whenever you see a relationship like

Current State

↓

Previous State

it usually indicates a recursive or dynamic programming solution. Examples include:

- Tree Depth
- Tree Height
- Distance to Root

- Ancestor Problems

Pattern 3 : Compute Once, Reuse Forever

Suppose `Depth(6)` has already been calculated. If another node needs it, should we calculate it again? No. Simply reuse the stored value. This is the classic idea of **Memoization**. Whenever you notice

"The same value may be needed multiple times."

always think about caching the result.

Pattern 4 : Longest Chain = Maximum Depth

Many tree problems eventually ask for

- Maximum Levels
- Longest Ancestor Chain
- Deepest Node
- Longest Path from Root

These are all variations of the same concept. Instead of counting children, focus on the longest path from the root. That path determines the answer.

Pattern 5 : Forest Problems

Some problems contain multiple independent trees. Example

Tree A

```
1
|
2
```

Tree B

```
3
|
4
|
5
```

Instead of solving only one tree, remember this rule:

Process every tree.

Combine the answers.

Depending on the problem, the final answer may be

- Maximum
- Minimum
- Sum
- Count

In this problem, we take the `Maximum Depth` among all trees.

Pattern 6 : Hidden Tree Problems

Not every problem says

"You are given a tree."

Sometimes, the tree is hidden inside the input. Common hints include:

- Every node has exactly one parent.
- One parent pointer is given for each node.
- A manager-employee relationship.
- Advisor-student relationship.
- Folder-parent folder structure.
- Family tree.
- Organization hierarchy.

Whenever you see these relationships, start thinking about trees.

Pattern 7 : Convert the Story into Mathematics

The story says Advisor and Advisee The mathematical meaning is
Ancestor and Descendant The story asks for Minimum Sessions The mathematical
meaning is Maximum Depth A strong Competitive Programmer always translates

Story

↓

Data Structure

↓

Algorithm

instead of solving based on the story itself.

Pattern Recognition Summary

Whenever you encounter a problem with these characteristics,

- ✓ Every node has one parent.
- ✓ Multiple roots may exist.
- ✓ Parent-child hierarchy.
- ✓ Longest ancestor chain matters.
- ✓ Same calculation is repeated.

your brain should immediately think

Tree / Forest

↓

Depth

↓

Recursion

↓

Memoization

↓

Maximum Depth

This is exactly the pattern used in this problem. The more problems you solve, the faster your brain will recognize this pattern automatically.

Bootcamp Summary

Let's summarize everything we learned from this problem.

What We Learned

This problem looked like a scheduling problem at first. However, after carefully analyzing the input, we discovered that it is actually a **Tree (Forest)** problem. The biggest lesson is:

Never solve the story. Solve the underlying data structure.

Core Observation

Every student has at most one advisor. This naturally forms a `Tree` or, if there are multiple roots, `Forest`. Once we recognize this structure, the problem becomes much simpler.

Key Formula

The depth of a node is defined as

$$\text{Depth}(\text{Node})$$

$$=$$

$$\text{Depth}(\text{Parent})$$

$$+$$

$$1$$

For a root node,

$$\text{Depth}(\text{Root})$$

$$=$$

$$1$$

This simple recursive relation is enough to solve the entire problem.

Main Idea

The minimum number of sessions is equal to Maximum Depth of the Forest Why?
 Because every student on the same ancestor chain must attend different sessions.
 Therefore, the longest chain determines the minimum number of sessions required.

Algorithm

Our solution follows these steps:

1. Read the parent of every student.

2. Calculate the depth of each student recursively.
3. Store every calculated depth using memoization.
4. Keep track of the maximum depth.
5. Print the maximum depth.

Complexity

Analysis	Complexity
Time Complexity	$O(N)$
Space Complexity	$O(N)$

Since every node is processed only once, this is the optimal solution.

Important Concepts Practiced

After solving this problem, you have practiced the following concepts:

- Forest Representation
- Parent Array
- Tree Depth
- Recursive Thinking
- Memoization
- Dynamic Programming on Trees (Basic Idea)
- Longest Ancestor Chain
- Complexity Analysis

Pattern to Remember

Whenever you see a problem where

- every node has exactly one parent,
- multiple roots may exist,
- and the answer depends on the longest ancestor chain,

immediately think:

Parent Array

↓

Tree / Forest

↓

Depth

↓

Memoization

↓

Maximum Depth

Recognizing this pattern quickly will help you solve many future problems involving trees and hierarchies.

Final Takeaway

This problem teaches an important Competitive Programming mindset:

The story may change, but the pattern remains the same.

Today it was about **students and advisors**. Tomorrow it might be

- employees and managers,

- folders and parent folders,
- family members,
- organizations,
- categories,
- or any hierarchical structure.

The names will change. The algorithm will not. If you can identify the hidden tree and compute its maximum depth efficiently, you can solve all such problems with confidence.

Problem 3 — Mission: Welcome

Problem Statement

A programming contest is being held in the Department of Computer Science and Engineering (CSE) at Shanto Mariam University of Creative Technology (SMUCT). As a participant, your first task is to display a welcome message. Print the following line exactly as shown: `WeLcome to the SMUCT CSE FEST V.3 2026`

Input Format

There is no input.

Constraints

Your program must print the required output exactly as specified.

Output Format

Print the following line exactly: `WeLcome to the SMUCT CSE FEST V.3 2026`

Sample Output

Welcome to the SMUCT CSE FEST V.3 2026

Solution (C)

```
#include <stdio.h>

int main()
{
    printf("Welcome to the SMUCT CSE FEST V.3 2026\n");

    return 0;
}
```

Problem 4 — K-Distinct Substrings

Problem Statement

You are given a string **S** consisting of lowercase English letters, and an integer **K**. Count the number of substrings of **S** that contain **exactly K distinct characters**. Two substrings are considered different if they occur at different **(start, end)** index pairs, even if the resulting characters are identical. Count the number of pairs (i, j) with $1 \leq i \leq j \leq N$ such that the substring $S[i..j]$ contains exactly K distinct characters. Since the answer can be very large, output it modulo **1,000,000,007**.

Input Format

- The first line contains an integer **N** — the length of the string.

- The second line contains a string **S** of length **N**, consisting of lowercase English letters.
- The third line contains an integer **K**.

Constraints

- $1 \leq N \leq 10^6$
- $1 \leq K \leq 26$
- S contains only lowercase English letters ('a'-'z').

Output Format

Print a single integer — the number of substrings containing exactly **K** distinct characters, modulo **1,000,000,007**.

Sample Input 0

```
5
abcba
2
```

Sample Output 0

```
5
```

Sample Input 1

```
6
aabbcc
2
```

Sample Output 1

8

Solution (C)

```

#include <stdio.h>
#include <string.h>

#define MAXN 1000005
#define MOD 1000000007LL

char s[MAXN];
int freq[26];

long long atMostKDistinct(int n, int k)
{
    if (k < 0)
        return 0;

    memset(freq, 0, sizeof(freq));

    int left = 0;
    int distinct = 0;
    long long count = 0;

    for (int right = 0; right < n; right++)
    {
        int c = s[right] - 'a';

        if (freq[c] == 0)
            distinct++;

        freq[c]++;

        while (distinct > k)
        {
            int x = s[left] - 'a';

            freq[x]--;

            if (freq[x] == 0)
                distinct--;
        }
    }
}

```

```

        left++;
    }

    count += (right - left + 1);
}

return count;
}

int main()
{
    int n, k;

    scanf("%d", &n);
    scanf("%s", s);
    scanf("%d", &k);

    long long ans =
        (atMostKDistinct(n, k) -
         atMostKDistinct(n, k - 1)) % MOD;

    if (ans < 0)
        ans += MOD;

    printf("%lld\n", ans);

    return 0;
}

```

Thinking Process

At first glance, this problem asks us to count substrings with **exactly K distinct characters**. A brute-force approach would be:

- Generate every possible substring.

- Count the number of distinct characters in each substring.
- If the count equals **K**, increase the answer.

For a string of length **N**, there are $N \times (N + 1) / 2$ possible substrings. Since **N** can be as large as **10⁶**, checking every substring is impossible. So we need a much faster idea.

Key Observation

Counting substrings with **exactly K** distinct characters directly is difficult. Instead, let's think about an easier problem. What if we count

Substrings having at most K distinct characters. This is much easier because once a window satisfies the condition, every smaller window inside it also satisfies the condition. That property allows us to use the **Sliding Window (Two Pointers)** technique.

The Magic Formula

Suppose

A = Number of substrings having at most K distinct characters

B = Number of substrings having at most (K - 1) distinct characters

Then

Exactly K

=

At Most K

-

At Most $(K - 1)$

This is the most important idea of this problem.

Why Does This Work?

Imagine every substring is grouped by the number of distinct characters.

At Most 3

contains

1 distinct

2 distinct

3 distinct

At Most 2

contains

1 distinct

2 distinct

If we subtract them,

At Most 3

-

At Most 2

=

Only 3 distinct

Similarly,

At Most K

-

At Most (K - 1)

=

Exactly K

This trick appears very frequently in Sliding Window problems.

How Do We Count "At Most K"?

We maintain a window [left right] Initially,

left = 0

right = 0

As we move **right**, we add one character into the window. If the number of distinct characters becomes larger than **K**, we move **left** forward until the window again contains at most **K** distinct characters. At every step, the current window is valid.

Counting Valid Substrings

Suppose after adjusting the window we have

`left = 3`

`right = 8`

The window is 3 4 5 6 7 8 Every substring ending at index **8** and starting anywhere between **3** and **8** is also valid.

`3 → 8`

`4 → 8`

`5 → 8`

`6 → 8`

`7 → 8`

`8 → 8`

Number of valid substrings

`=`

`right - left + 1`

So after processing every position, we simply add `right - left + 1` to the answer.

Overall Plan

1. Write a function to count substrings having **at most K** distinct characters.
2. Call it once for **K**.
3. Call it again for **K - 1**.
4. Subtract the two results.
5. Print the final answer modulo **$10^9 + 7$** .

This transforms a difficult counting problem into two simple Sliding Window computations, each running in linear time.

Building the Algorithm

Now let's convert our idea into an algorithm.

Step 1 : Create a Sliding Window

We maintain two pointers.

```

left           right
  ↓             ↓
[a b c b a]
```

Initially,

```

left = 0
right = 0
```

The window always represents `s[left ... right]`

Step 2 : Expand the Window

Move the **right** pointer one character at a time. Whenever a new character enters the window, increase its frequency. `freq[S[right]]++` If this character appears for the first time, increase `distinct++`

Step 3 : Keep the Window Valid

Our window should never contain more than **K** distinct characters. If `distinct > K` we repeatedly move the **left** pointer.

```
while (distinct > K)
{
    Remove S[left]

    left++
}
```

Whenever the frequency of a character becomes zero, that character completely disappears from the window.

```
freq[ch]--

if(freq[ch] == 0)
    distinct--
```

Eventually, `distinct ≤ K` again, making the window valid.

Step 4 : Count All Valid Substrings

Suppose the window becomes

`left = 2`

`right = 6`

Index

0 1 2 3 4 5 6

L
R
↓
↓

a b c b a c b

Every substring ending at **right** and starting anywhere from **left** to **right** is valid. Those substrings are

`2 → 6`

`3 → 6`

`4 → 6`

`5 → 6`

`6 → 6`

Total `right - left + 1` So after processing every position, we do
`answer += right - left + 1`

Step 5 : Build the Function

We create a function

```
long long atMostKDistinct(int n, int k)
```

This function returns

```
Number of substrings
having at most K distinct characters.
```

Inside the function we

- initialize the frequency array,
- maintain the sliding window,
- count all valid substrings,
- return the total count.

Step 6 : Compute the Final Answer

From our observation,

Exactly K

=

At Most K

-

At Most (K - 1)

So in the main function, we simply write

```
answer =  
atMostKDistinct(n, K)  
-  
atMostKDistinct(n, K - 1);
```

Since the problem asks for the answer modulo $10^9 + 7$ we compute

```
answer %= MOD;  
  
if (answer < 0)  
    answer += MOD;
```

Finally, print the answer.

Complete Algorithm

Read N , S and K

↓

Compute $\text{AtMost}(K)$

↓

Compute $\text{AtMost}(K-1)$

↓

Subtract the two values

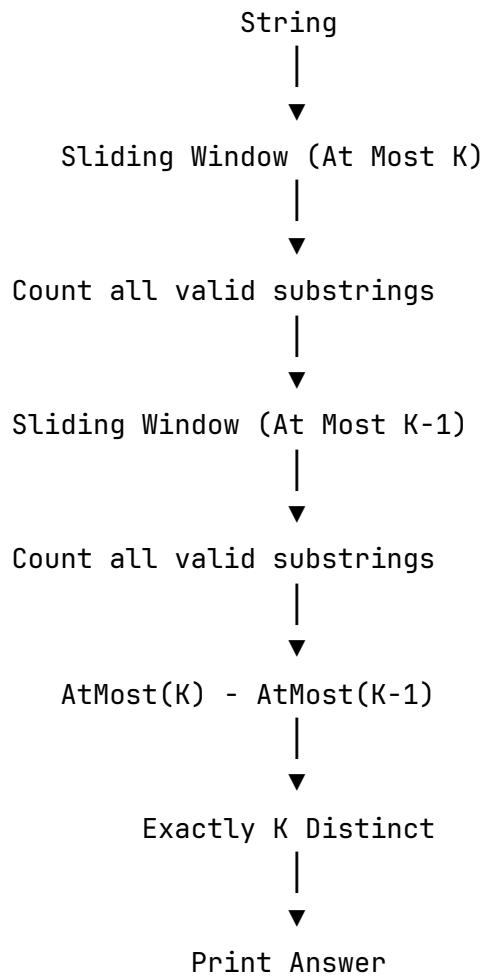
↓

Take modulo

↓

Print the answer

Visual Flow



This algorithm processes the string only twice, and each pass is linear, making it efficient even for the maximum constraint of **$N = 10^6$** .

Dry Run

Let's understand the algorithm step by step using the sample input.

Sample Input

`N = 5`

`S = "abcba"`

`K = 2`

Our algorithm computes

`Exactly(2)`

`=`

`AtMost(2)`

`-`

`AtMost(1)`

So we first calculate `AtMost(2)` and then `AtMost(1)`.

Dry Run of AtMost(2)

Initially

`left = 0`

`distinct = 0`

`answer = 0`

Frequency array is empty.

Iteration 1

```
right = 0
```

```
Character = 'a'
```

Window [a] Distinct characters 1 Valid window. New valid substrings ending here
"a" Count added

```
right - left + 1
```

```
=
```

```
0 - 0 + 1
```

```
=
```

```
1
```

Current answer 1

Iteration 2

```
right = 1
```

```
Character = 'b'
```

Window [a b] Distinct 2 Still valid. Valid substrings ending here

"ab"

"b"

Added 2 Current answer $1 + 2 = 3$

Iteration 3

right = 2

Character = 'c'

Window [a b c] Distinct 3 Too many. Shrink from the left. Remove 'a' Window becomes [b c] Distinct becomes 2 Valid again. Valid substrings ending here

"bc"

"c"

Added 2 Current answer 5

Iteration 4

right = 3

Character = 'b'

Window [b c b] Distinct 2 Still valid. Valid substrings ending here

"bcb"

"cb"

"b"

Added 3 Current answer 8

Iteration 5

right = 4

Character = 'a'

Window [b c b a] Distinct 3 Too many. Remove 'b' Window [c b a] Still 3
Remove 'c' Window [b a] Distinct becomes 2 Now valid. Valid substrings ending here

"ba"

"a"

Added 2 Final

AtMost(2)

=

10

Dry Run of AtMost(1)

Now repeat the same process with $K = 1$ Only windows having one distinct character are allowed. Valid substrings are

"a"

"b"

"c"

"b"

"a"

Total

AtMost(1)

=

5

Final Calculation

Using our formula,

Exactly(2)

=

AtMost(2)

-

AtMost(1)

Substitute the values,

Exactly(2)

=

10

-

5

=

5

Therefore, Answer = 5 which matches the sample output.

What Actually Happened?

Instead of trying to count substrings with exactly **2** distinct characters directly, we counted

```
All substrings  
having at most 2 distinct characters.
```

Then we removed

```
All substrings  
having at most 1 distinct character.
```

The remaining substrings are exactly those containing **2 distinct characters**. This elegant trick allows us to solve the problem in linear time using the Sliding Window technique.

Complexity Analysis

Time Complexity

The function

```
atMostKDistinct()
```

uses the **Sliding Window (Two Pointers)** technique. Although there are two pointers (`left` and `right`), neither of them ever moves backward.

Right Pointer

The **right** pointer starts from the beginning of the string and moves to the end.

$0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow N-1$ So it moves exactly N times.

Left Pointer

Whenever the window contains more than **K** distinct characters, the **left** pointer moves forward. Just like the right pointer, it never moves backward. $0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow N-1$

Therefore, the total movement of the left pointer is also at most N

Total Work

Every character

- enters the window once,
- leaves the window once.

So the total number of operations is proportional to

$$N + N$$

$$=$$

$$2N$$

Ignoring constant factors,

Time Complexity

=

$O(N)$

Since we call

`atMostKDistinct()`

twice,

$O(N)$

+

$O(N)$

=

$O(2N)$

=

$O(N)$

Thus, the overall time complexity is $O(N)$

Space Complexity

We maintain only a few additional variables. The only extra data structure is

```
int freq[26];
```

Since the string contains only lowercase English letters, the frequency array always has a fixed size of 26 regardless of the value of **N**. Therefore,

Extra Space

=

$O(26)$

=

$O(1)$

Complexity Summary

Analysis	Complexity
Time Complexity	$O(N)$
Space Complexity	$O(1)$

This is the optimal solution for the given constraints because every character of the string is processed only a constant number of times.

Common Mistakes

While solving this problem, beginners often make the following mistakes.

Mistake 1 : Trying to Count "Exactly K" Directly

Many beginners attempt to maintain a window with exactly **K** distinct characters. For example,

Window

[a b c]

contains exactly 3 distinct characters. They count it and immediately move the window. Unfortunately, this approach misses many valid substrings. The Sliding Window technique naturally works with At Most K not Exactly K Always remember the formula

Exactly K

=

At Most K

-

At Most (K - 1)

Mistake 2 : Forgetting to Decrease the Distinct Count

Suppose the current frequencies are

a → 2

b → 1

c → 1

After removing **b** the frequencies become

a → 2

b → 0

c → 1

Since **b** no longer exists in the window, you must decrease

```
distinct--;
```

Many beginners forget this step, causing the window to think that **b** is still present.

Mistake 3 : Using an Incorrect Window Condition

Some write

```
while (distinct ≥ K)
```

This is incorrect. We are allowed to have Exactly **K** distinct characters inside the window. The correct condition is

```
while (distinct > K)
```

Only when the number of distinct characters exceeds **K** should we shrink the window.

Mistake 4 : Counting Only One Substring

After finding a valid window, some beginners do

```
answer++;
```

This is incorrect. If

```
left = 2
```

```
right = 6
```

then all of these substrings are valid.

```
2 → 6
```

```
3 → 6
```

```
4 → 6
```

```
5 → 6
```

```
6 → 6
```

So we must add

```
answer += right - left + 1;
```

instead of

```
answer++;
```

Mistake 5 : Forgetting the Edge Case (K = 0)

Our formula uses $\text{AtMost}(K - 1)$. If $K = 0$ then we need $\text{AtMost}(-1)$. No substring can contain fewer than zero distinct characters. So the function should immediately return

```
0
```

Our solution safely handles this by checking

```
if (k < 0)
    return 0;
```

Mistake 6 : Using Brute Force

A common first idea is

Generate every substring

↓

Count distinct characters

↓

Check if it equals K

This requires $O(N^2)$ or even $O(N^3)$ depending on the implementation. With $N \leq 10^6$ such approaches are far too slow. Sliding Window reduces the complexity to $O(N)$

Mistake 7 : Forgetting the Frequency Array

Some beginners try to determine whether a character already exists in the window by scanning the entire substring repeatedly. This makes each operation much slower. Instead, maintain a frequency array.

```
int freq[26];
```

Now every insertion and removal takes only $O(1)$ time.

Mistake 8 : Integer Overflow

The number of substrings can be very large. For example, $N = 1,000,000$ The total number of substrings is approximately

$$N \times (N + 1) / 2$$

$$\approx 5 \times 10^{11}$$

This value cannot fit inside an

`int`

Always use

`long long`

for the answer.

Common Mistakes Summary

Before submitting your solution, verify the following checklist:

- ✓ Used the Sliding Window technique.
- ✓ Counted `AtMost(K)` correctly.
- ✓ Counted `AtMost(K-1)` correctly.
- ✓ Applied:
 $\text{Exactly}(K) = \text{AtMost}(K) - \text{AtMost}(K-1)$
- ✓ Used
 `answer += right - left + 1`
- ✓ Decreased distinct when a frequency becomes zero.
- ✓ Used `while(distinct > K)`.
- ✓ Stored the answer in `long long`.
- ✓ Returned 0 when $K < 0$.

If all the above points are satisfied, your solution will correctly handle every valid test case.

Pattern Recognition

One of the most important skills in Competitive Programming is recognizing the underlying pattern of a problem. At first glance, this problem appears to be about **counting substrings**. However, the real pattern is much deeper.

Pattern 1 : Sliding Window (Two Pointers)

The problem asks us to examine **contiguous substrings**. Whenever we need to process contiguous segments efficiently while maintaining certain properties, the **Sliding Window** technique should come to mind. In this problem, our window is `[left ..... right]` which continuously expands and shrinks while maintaining $\text{Number of distinct characters} \leq K$. This allows us to process the entire string in linear time.

Pattern 2 : Frequency Array

To know whether a character is entering or leaving the window, we maintain a frequency array.

```
int freq[26];
```

This allows us to

- insert a character in **$O(1)$** ,
- remove a character in **$O(1)$** ,
- update the number of distinct characters efficiently.

Whenever the alphabet size is small and fixed, using a frequency array is usually the best choice.

Pattern 3 : At Most → Exactly

This is the key observation of the entire problem. Instead of counting Exactly K directly, we convert it into

$$\text{Exactly}(K)$$

$$=$$

$$\text{AtMost}(K)$$

$$-$$

$$\text{AtMost}(K-1)$$

Many difficult counting problems become much easier after applying this transformation. Whenever you encounter **"exactly K"**, consider whether you can first count **"at most K"** and then subtract the cases with **"at most K-1"**.

Pattern 4 : Counting All Valid Windows

After adjusting the window, every starting position inside the current window forms a valid substring ending at the current **right** pointer. Therefore, instead of counting only one substring, we add $\text{right} - \text{left} + 1$. This counting trick is one of the most common techniques in Sliding Window problems.

Pattern Recognition Checklist

When solving future problems, ask yourself the following questions:

- ☐ Does the problem involve contiguous subarrays or substrings?
- ☐ Can I maintain a valid window using two pointers?
- ☐ Can I track the required information using a frequency array?
- ☐ Does the problem ask for "Exactly K"?
- ☐ Can I convert it into
AtMost(K) - AtMost(K-1)?
- ☐ Can every valid window contribute
(right - left + 1)
substrings?

If most of the answers are **Yes**, the Sliding Window approach is likely the correct solution.

Similar Problems

The same ideas appear in many well-known Competitive Programming problems, such as:

- Longest Substring Without Repeating Characters
- Longest Substring with At Most K Distinct Characters
- Count Binary Subarrays with Sum = Goal
- Subarrays with K Different Integers
- Fruit Into Baskets
- Minimum Window Substring

Mastering this problem will make many other Sliding Window problems much easier to solve.

Bootcamp Summary

What We Learned

In this problem, we learned that counting substrings with **exactly K distinct characters** directly is difficult. Instead of solving the problem head-on, we transformed it into a much simpler one. The key idea was

Exactly(K)

=

AtMost(K)

-

AtMost(K-1)

This observation allowed us to solve the problem efficiently using the **Sliding Window (Two Pointers)** technique.

Key Concepts

By solving this problem, you practiced the following important concepts:

- Sliding Window (Two Pointers)
- Frequency Array
- Counting Distinct Characters
- Dynamic Window Expansion and Shrinking
- Counting Valid Substrings
- Mathematical Transformation (Exactly K = AtMost K - AtMost (K-1))
- Linear-Time String Processing

Important Formula

Always remember this powerful counting trick:

$\text{Exactly}(K)$

=

$\text{AtMost}(K)$

-

$\text{AtMost}(K-1)$

This formula appears in many Competitive Programming problems involving counting subarrays or substrings.

Sliding Window Workflow

The overall process is:

Expand Right Pointer



Add Current Character



Update Frequency



If Distinct > K

Shrink from Left



Window Becomes Valid



Add

(right - left + 1)



Continue

Complexity

Analysis	Complexity
Time Complexity	O(N)

Analysis	Complexity
Space Complexity	O(1)

Even for very large strings, the algorithm remains efficient because every character enters and leaves the window at most once.

Takeaway

Whenever you encounter a problem involving:

- contiguous subarrays or substrings,
- a condition based on **at most K** elements,
- or counting **exactly K** occurrences or distinct values,

think about using a **Sliding Window** and ask yourself whether the problem can be transformed using

Exactly(K)

=

AtMost(K)

-

AtMost(K-1)

Recognizing this pattern can turn an otherwise difficult problem into a simple linear-time solution.

Problem 5 — The Treasure Caravan

Problem Statement

A caravan travels through checkpoints containing treasure. There are N checkpoints numbered from 1 to N . The i -th checkpoint contains A_i units of treasure, which may be negative. The caravan starts at checkpoint 1 and must reach checkpoint N . From checkpoint i , it can move to:

- $i + 1$
- $i + 2$
- $i + K$, where K is the given integer.

Determine the **maximum total treasure** that can be collected upon reaching checkpoint N .

Input Format

The first line contains an integer T — the number of test cases. For each test case:

- The first line contains two integers N and K .
- The second line contains N integers:

$A_1, A_2, A_3, \dots, A_n$.

Constraints

- $1 \leq T \leq 10$
- $2 \leq N \leq 200000$
- $2 \leq K \leq N$
- $-10^9 \leq A_i \leq 10^9$

Output Format

For each test case, print one integer — the maximum treasure obtainable.

Sample Input 0

```
3
8 4
6 -2 9 -7 2 -1 8 -4
5 3
10 -20 5 -20 15
12 5
-19 -3 6 5 23 -19 -27 13 3 22 -30
```

Sample Output 0

```
21
30
39
```

Explanation 0

Test Case 1 The optimal path is $1 \rightarrow 3 \rightarrow 5 \rightarrow 7 \rightarrow 8$ Collected treasure

$$6 + 9 + 2 + 8 + (-4)$$

=

21

Test Case 2 The optimal path is $1 \rightarrow 3 \rightarrow 5$ Collected treasure

$$10 + 5 + 15$$

=

$$30$$

Test Case 3 The optimal path is 1 → 3 → 4 → 5 → 10 → 12 Collected treasure

$$-19 + 6 + 5 + 23 + 22 + 2$$

=

$$39$$

Sample Input 1

1

5 5

1 -100 2 -100 5

Sample Output 1

6

Explanation 1

$N = 5, K = 5$. The $+K$ jump requires $i + 5 \leq 5$ ($i \leq 0$), which is never valid, so effectively only the $+1$ and $+2$ moves exist. The best path avoiding the -100 checkpoints is $1 \rightarrow 3 \rightarrow 5$: $A_1 + A_3 + A_5 = 1 + 2 + 3 = 6$.

Solution (C)

```

#include <stdio.h>

#define MAXN 200005
#define NEG_INF -(1LL << 60)

long long a[MAXN];
long long dp[MAXN];

long long max(long long x, long long y)
{
    return (x > y) ? x : y;
}

int main()
{
    int T;
    scanf("%d", &T);

    while (T--)
    {
        int n, k;
        scanf("%d %d", &n, &k);

        for (int i = 1; i ≤ n; i++)
            scanf("%lld", &a[i]);

        for (int i = 1; i ≤ n; i++)
            dp[i] = NEG_INF;

        dp[1] = a[1];

        for (int i = 1; i ≤ n; i++)
        {
            if (dp[i] == NEG_INF)
                continue;

            if (i + 1 ≤ n)

```

```

        dp[i + 1] = max(dp[i + 1], dp[i] + a[i + 1]);

        if (i + 2 ≤ n)
            dp[i + 2] = max(dp[i + 2], dp[i] + a[i + 2]);

        if (i + k ≤ n)
            dp[i + k] = max(dp[i + k], dp[i] + a[i + k]);
    }

    printf("%lld\n", dp[n]);
}

return 0;
}

```

Thinking Process

1. What is the problem asking?

At first glance, this looks like a path-finding problem. We have

- **N checkpoints**
- each checkpoint contains some treasure (possibly negative),
- we start from checkpoint **1**,
- and we must reach checkpoint **N**.

At every checkpoint, we have exactly three possible moves:

+1

+2

+K

Our goal is to collect the **maximum total treasure**.

2. First Observation

Suppose

$N = 8$

$K = 4$

and $6 \ -2 \ 9 \ -7 \ 2 \ -1 \ 8 \ -4$ The checkpoints look like

1	2	3	4	5	6	7	8
6	-2	9	-7	2	-1	8	-4

From checkpoint 1 we can go to

2

3

5

because

1+1

1+2

1+4

are valid.

3. What Should We Optimize?

Suppose we arrive at checkpoint 5. There may be many different paths that reach it. Example: $1 \rightarrow 3 \rightarrow 5$ or $1 \rightarrow 2 \rightarrow 3 \rightarrow 5$ or $1 \rightarrow 5$. Some paths collect more treasure than others. When we stand at checkpoint 5, do we care about every path? No. We only care about

the maximum treasure collected so far.

Everything else is useless.

4. A State Begins to Appear

Whenever we only need the **best value** for every position, Dynamic Programming should come to mind. Let's define $dp[i]$ as

Maximum treasure that can be collected upon reaching checkpoint i .

Now every checkpoint has exactly one answer.

5. How Can We Reach a Checkpoint?

Consider checkpoint i . How can we arrive here? Only three possibilities exist. We came from $i-1$ or $i-2$ or $i-K$. No other move is allowed. Therefore, $dp[i]$ depends only on these previous checkpoints.

6. Transition Idea

Suppose $dp[i]$ is already known. From here, we can move to

$i+1$

$i+2$

$i+K$

Whenever we move, the treasure at the destination is added. For example,

```
dp[i+1]
=
max(
  dp[i+1],
  dp[i] + A[i+1]
)
```

The same idea works for $i+2$ and $i+K$

7. Initial State

Where does the caravan start? Always at Checkpoint 1 Therefore,

$dp[1]$

=

$A[1]$

Every other checkpoint is initially unreachable. So we initialize them with $-\infty$ meaning Impossible to reach yet.

8. Why Does This Work?

Imagine processing checkpoints from left to right. Whenever we know the best value at one checkpoint, we immediately use it to improve every checkpoint that can be reached from it. Eventually, every reachable checkpoint stores the maximum treasure obtainable there. Since we must finish at Checkpoint N our answer is simply $dp[N]$

9. Brute Force vs DP

A brute-force solution would try every possible path. At every checkpoint, there are up to 3 choices. The number of paths grows exponentially.

$3 \times 3 \times 3 \times \dots$

$\approx 3^N$

which is impossible for $N = 200000$. Dynamic Programming avoids recomputing the same checkpoint repeatedly. Each checkpoint is processed only once, making the solution efficient.

10. Hidden Observation

This problem may look like a graph traversal problem, because checkpoints and moves naturally form a directed graph. However, every edge always goes

Left

↓

Right

There are no cycles. This means the graph is actually a **Directed Acyclic Graph (DAG)**. Processing checkpoints from $1 \rightarrow N$ already follows a topological order. That allows us to replace graph algorithms with a simple Dynamic Programming solution.

Building the Algorithm

Now that we know $dp[i]$ represents the maximum treasure obtainable upon reaching checkpoint i , let's convert the idea into an algorithm.

Step 1 : Define the DP Array

We create an array

`dp[i]`

where

`dp[i]`

=

Maximum treasure collected
when we reach checkpoint i .

Initially,

`dp[1]`

=

`A[1]`

because the caravan starts there. Every other position is initialized as $-\infty$ which means
This checkpoint has not been reached yet.

Step 2 : Process Checkpoints from Left to Right

We iterate

```
for(i = 1; i ≤ N; i++)
```

Suppose $dp[i]$ has already been computed. From checkpoint i we can make three possible moves.

Step 3 : Move by +1

If $i + 1 \leq N$ then

```
dp[i+1]
=
max(
  dp[i+1],
  dp[i] + A[i+1]
)
```

Graphically, $i \longrightarrow i+1$ The treasure at $i+1$ is added to the current best value.

Step 4 : Move by +2

Similarly, if $i + 2 \leq N$ then

```

dp[i+2]

=

max(
  dp[i+2],
  dp[i] + A[i+2]
)

```

Graphically, $i \longrightarrow i+2$ Again, we keep only the better path.

Step 5 : Move by +K

Finally, if $i + K \leq N$ then

```

dp[i+K]

=

max(
  dp[i+K],
  dp[i] + A[i+K]
)

```

Graphically, $i \longrightarrow i+K$ This jump allows us to skip many checkpoints, which is especially useful when several consecutive checkpoints contain negative treasure.

Step 6 : Ignore Unreachable States

Suppose

$dp[i]$

$=$

$-\infty$

This means Checkpoint i cannot be reached. Trying to extend such a path is meaningless. Therefore, before processing a checkpoint, we check

```
if(dp[i] == NEG_INF)
    continue;
```

This avoids unnecessary work.

Step 7 : Continue Until the End

Repeat the same process for every checkpoint. Whenever a better path reaches a checkpoint, its DP value is updated. Eventually, $dp[N]$ contains the maximum treasure obtainable upon reaching the last checkpoint.

Complete Algorithm

Read T

↓

For every test case

↓

Read N and K

↓

Read the treasure array

↓

Initialize DP with $-\infty$

↓

$dp[1] = A[1]$

↓

For every checkpoint

Update

$i+1$

$i+2$

$i+K$

↓

Print dp[N]

Visual Flow

Checkpoint 1



Compute dp[1]



Update

i+1

i+2

i+K



Process Next Checkpoint



Repeat Until N



Answer = $dp[N]$

Why Does This Algorithm Work?

Every valid path reaching a checkpoint must come from exactly one of

$i-1$

$i-2$

$i-K$

Our algorithm explores every legal move exactly once. Whenever multiple paths reach the same checkpoint, we store only the one with the greatest treasure. Since checkpoints are processed from left to right, all possible predecessors of a checkpoint have already been considered before it is used to update future checkpoints. Thus, every checkpoint eventually stores the maximum treasure that can be collected upon reaching it. Therefore, $dp[N]$ is exactly the maximum treasure obtainable when the caravan reaches the final checkpoint.

Dry Run

Let's execute the algorithm step by step using the first sample.

Sample Input

N = 8
K = 4

Treasure:

6 -2 9 -7 2 -1 8 -4

Step 1 : Initialization

Initially, $dp[1] = 6$ Every other checkpoint is unreachable.

Index

1 2 3 4 5 6 7 8

DP

6 $-\infty$ $-\infty$ $-\infty$ $-\infty$ $-\infty$ $-\infty$ $-\infty$

Step 2 : Process Checkpoint 1

Current $dp[1] = 6$ Possible moves

1 → 2

1 → 3

1 → 5

Updates

dp[2]

=

6 + (-2)

=

4

dp[3]

=

6 + 9

=

15

$dp[5]$

$=$

$6 + 2$

$=$

8

Now 6 4 15 $-\infty$ 8 $-\infty$ $-\infty$ $-\infty$

Step 3 : Process Checkpoint 2

Current $dp[2] = 4$ Moves

$2 \rightarrow 3$

$2 \rightarrow 4$

$2 \rightarrow 6$

Update

$dp[3]$

$=$

$\max(15,$
 $4+9)$

$=$

15

No improvement.

$dp[4]$

$=$

$4 + (-7)$

$=$

-3

$dp[6]$

$=$

$4 + (-1)$

$=$

3

DP becomes 6 4 15 -3 8 3 $-\infty$ $-\infty$

Step 4 : Process Checkpoint 3

Current $dp[3]=15$ Moves

3 $\rightarrow$ 4

3 $\rightarrow$ 5

3 $\rightarrow$ 7

Update

$dp[4]$

$=$

$\max(-3,$
 $15-7)$

$=$

8

$dp[5]$

$=$

$\max(8,$
 $15+2)$

$=$

17

$dp[7]$

$=$

$15+8$

$=$

23

Now 6 4 15 8 17 3 23 $-\infty$

Step 5 : Process Checkpoint 4

Current $dp[4]=8$ Moves

4 $\rightarrow$ 5

4 $\rightarrow$ 6

4 $\rightarrow$ 8

Updates

`dp[5]`

`=`

`max(17,`
`8+2)`

`=`

`17`

`dp[6]`

`=`

`max(3,`
`8-1)`

`=`

`7`

$dp[8]$

$=$

$8-4$

$=$

4

DP 6 4 15 8 17 7 23 4

Step 6 : Process Checkpoint 5

Current $dp[5]=17$ Moves

$5 \rightarrow 6$

$5 \rightarrow 7$

(+K is outside the array.) Updates

`dp[6]`

`=`

`max(7,
17-1)`

`=`

`16`

`dp[7]`

`=`

`max(23,
17+8)`

`=`

`25`

DP 6 4 15 8 17 16 25 4

Step 7 : Process Checkpoint 6

Current `dp[6]=16` Moves

6 → 7

6 → 8

Updates

dp[7]

=

max(25,
16+8)

=

25

dp[8]

=

max(4,
16-4)

=

12

DP 6 4 15 8 17 16 25 12

Step 8 : Process Checkpoint 7

Current $dp[7]=25$ Move $7 \rightarrow 8$ Update

$dp[8]$

=

$\max(12,$
 $25-4)$

=

21

Final DP 6 4 15 8 17 16 25 21

Step 9 : Process Checkpoint 8

Checkpoint 8 is the destination. No further moves exist. The answer is

$dp[8]$

=

21

which matches the sample output.

Visualizing the Best Path

The optimal updates came from

1

↓

3

↓

5

↓

7

↓

8

Collected treasure

6

↓

9

↓

2

↓

8

↓

-4

Total

 $6 + 9 + 2 + 8 - 4$

=

21

Why Didn't We Store Every Path?

Notice that checkpoint 5 could be reached in several ways. For example, $1 \rightarrow 5$ gives 8 while $1 \rightarrow 3 \rightarrow 5$ gives 17. Instead of remembering both, we store only

```
dp[5]
```

```
=
```

```
17
```

because any future path starting from checkpoint 5 will always be better if it starts with the maximum treasure collected so far. This is the key optimization that makes Dynamic Programming efficient.

Complexity Analysis

Whenever we solve a Dynamic Programming problem, we should analyze two things:

1. **Time Complexity**
2. **Space Complexity**

Let's analyze both.

Time Complexity

For every test case, we process the checkpoints one by one.

```
for(i = 1; i ≤ N; i++)
```

For each checkpoint, we perform at most **three** updates.

$$i + 1$$

$$i + 2$$

$$i + K$$

Each update consists of only a few constant-time operations.

- Boundary check
- Addition
- Comparison
- Assignment

Therefore, the work done for one checkpoint is $O(1)$. Since there are N checkpoints, the total work becomes $O(N)$ for one test case.

If there are T test cases, the overall complexity is $O(\Sigma N)$ where ΣN is the total number of checkpoints across all test cases.

Why Isn't It Exponential?

A beginner might think

"At every checkpoint we have three choices."

That is true. A brute-force search would indeed explore

$$3 \times 3 \times 3 \times \dots$$

$$\approx 3^N$$

possible paths, which is impossible even for $N = 50$ let alone $N = 200000$. Dynamic Programming avoids this explosion. Instead of exploring every path, it computes the best answer for each checkpoint exactly once. After $dp[i]$ has been calculated, it is reused whenever needed. Thus, the exponential complexity becomes linear.

Space Complexity

We store two arrays. The treasure array

`a[MAXN]`

requires $O(N)$ memory.

The DP array

`dp[MAXN]`

also requires $O(N)$ memory.

Apart from these, we use only a few variables such as

`i`

`n`

`k`

which take constant space. Therefore, the total memory usage is $O(N)$

Can the DP Be Optimized?

In many DP problems, only the previous few states are needed, allowing constant-memory optimization. However, here every checkpoint may update $i + K$ where K can be as large as N . Future checkpoints may depend on values computed much earlier. Therefore, we must keep the entire DP array. The $O(N)$ memory usage is already optimal.

Final Complexity

Analysis	Complexity
Time Complexity	$O(N)$ per test case
Overall Time	$O(\Sigma N)$
Space Complexity	$O(N)$

This is optimal because every checkpoint must be processed at least once, making linear time the best possible complexity.

Common Mistakes

Even though the algorithm is straightforward, beginners often make a few common mistakes. Let's look at them one by one.

Mistake 1 : Initializing Every DP State to 0

Some students write

```
for(int i = 1; i ≤ N; i++)
    dp[i] = 0;
```

This is incorrect. Why? Because a checkpoint that has **never been reached** would still appear to have collected 0 treasure. The algorithm might then use this invalid state to update future checkpoints.

Correct Approach

Initialize every checkpoint as $-\infty$ (or a very small number) and only set

```
dp[1] = A[1];
```

This clearly distinguishes **reachable** and **unreachable** states.

Mistake 2 : Forgetting Boundary Checks

Suppose $i = N$ If we blindly execute

```
dp[i + 1]
```

or

```
dp[i + K]
```

the program accesses memory outside the array. This leads to

- Runtime Error
- Undefined Behavior
- Wrong Answer

Correct Approach

Always verify

```
if(i + 1 ≤ N)
```

```
if(i + 2 ≤ N)
```

```
if(i + K ≤ N)
```

before updating.

Mistake 3 : Updating from an Unreachable Checkpoint

Imagine $dp[i] = -\infty$ This means Checkpoint i cannot be reached. If we still execute

```
dp[i] + A[next]
```

the value is meaningless.

Correct Approach

Skip unreachable states.

```
if(dp[i] == NEG_INF)
    continue;
```

Only reachable checkpoints should generate new transitions.

Mistake 4 : Using `int` Instead of `long long`

The treasure values may be large, and many checkpoints are added together. Using

```
int
```

may cause integer overflow.

Correct Approach

Store both

```
a[]
```

and

```
dp[]
```

as

long long

to safely handle large sums.

Mistake 5 : Forgetting That Treasure Can Be Negative

Some beginners assume $Treasure \geq 0$ and try to skip negative checkpoints. However, negative values are part of the problem. Sometimes, crossing a negative checkpoint is unavoidable. Example 5 -10 20 The only valid path may include -10. Your algorithm must still compute the maximum possible total.

Mistake 6 : Thinking Greedy Will Work

A tempting strategy is:

"Always jump to the checkpoint with the largest immediate treasure."

This is not always optimal. Example 5 100 -1000 100 A locally attractive move may prevent reaching a much better overall path. The best decision depends on **future possibilities**, not just the current checkpoint. That's why Dynamic Programming is required.

Mistake 7 : Assuming Every Checkpoint Is Reachable

Depending on N and K , some checkpoints may never be visited. Your program should not assume that every $dp[i]$ has a valid value. Always check whether the state is reachable before extending it.

Mistake 8 : Forgetting to Reinitialize for Each Test Case

If there are multiple test cases, the DP array still contains values from the previous test. This leads to incorrect answers.

Correct Approach

Before solving each test case, reset the DP array.

```
for(int i = 1; i ≤ N; i++)  
    dp[i] = NEG_INF;
```

Then initialize

```
dp[1] = A[1];
```

Summary of Common Mistakes

Mistake	Correct Solution
Initialize DP with 0	Initialize with $-\infty$
Ignore array bounds	Check $i+1$, $i+2$, $i+K$ before updating
Update unreachable states	Skip when $dp[i] = \text{NEG_INF}$
Use <code>int</code> for large sums	Use <code>long long</code>
Ignore negative treasures	Handle them normally
Apply a greedy strategy	Use Dynamic Programming
Assume all checkpoints are reachable	Verify reachability
Forget to reset DP for each test case	Reinitialize before every test

By avoiding these mistakes, your implementation becomes both **correct** and **robust**, even for the largest test cases.

Pattern Recognition

One of the most important skills in Competitive Programming is recognizing **which algorithmic pattern** a problem belongs to. Let's identify the pattern used in **The Treasure Caravan**.

Step 1 : Identify the State

Whenever you see a problem asking questions like

- Maximum value

- Minimum value
- Number of ways
- Best score
- Longest path
- Shortest cost

for **every position**, think about Dynamic Programming. Here, the question is

"What is the maximum treasure we can have when reaching checkpoint i ?"

That immediately suggests a DP state.

$dp[i]$

=

Maximum treasure collected upon reaching checkpoint i

Step 2 : Does the Current State Depend on Previous States?

Look carefully at the allowed moves. From checkpoint i we can move to

$i+1$

$i+2$

$i+K$

Equivalently, checkpoint i can only be reached from

$i-1$

$i-2$

$i-K$

Therefore, each state depends on a fixed number of previous states. This is a classic sign of **1-D Dynamic Programming**.

Step 3 : Is Greedy Possible?

Ask yourself:

Can I always make the best immediate move?

No. A checkpoint with high treasure now may force us through several negative checkpoints later. Sometimes choosing a smaller reward now leads to a much larger reward overall. Since local decisions do not guarantee a globally optimal answer, **Greedy fails**.

Step 4 : Is Graph Traversal Needed?

If we draw the checkpoints as nodes,

1 → 2

↓

3

↓

5

we obtain a graph. However, every edge always goes

Left

↓

Right

There are **no backward edges**. Therefore, the graph contains **no cycles**. A graph with no cycles is called a **Directed Acyclic Graph (DAG)**. Instead of running graph algorithms, we can simply process the checkpoints from left to right, which is already a valid topological order.

Step 5 : Is This a Shortest/Longest Path Problem?

Conceptually, yes. Each checkpoint is a node, and each move is a directed edge. We want the path with the **maximum collected treasure**. Since the graph is a DAG, Dynamic Programming naturally computes this longest-path value in linear time.

Similar Problems

Once you understand this problem, you will notice the same pattern in many classic problems. Examples include:

- **Frog Jump** (minimum cost to reach the last stone)
- **Climbing Stairs** (number of ways to reach the top)
- **Minimum Cost Climbing Stairs**
- **House Robber**
- **Longest Increasing Subsequence** (different transition, same DP idea)
- **Coin Change** (state transition DP)

Although the stories are different, the underlying approach is the same:

1. Define a state.
2. Find transitions.
3. Compute answers iteratively.

Pattern Checklist

Whenever you encounter a new problem, ask yourself: ✓ Does the answer for one position depend on previous positions? ✓ Can I define a DP state? ✓ Is there an optimal substructure? ✓ Are overlapping subproblems present? ✓ Can I compute states in order? If most of these answers are **Yes**, the problem is very likely a **Dynamic Programming** problem.

Pattern Summary

Feature	Observation
Problem Type	Dynamic Programming
DP Dimension	1-D DP
State	$dp[i]$ = Maximum treasure upon reaching checkpoint i
Transition	From i , update $i+1$, $i+2$, and $i+k$
Processing Order	Left to Right
Graph Interpretation	Directed Acyclic Graph (DAG)
Time Complexity	$O(N)$
Space Complexity	$O(N)$

Key Takeaway

The story talks about **treasure**, **caravans**, and **checkpoints**, but the real problem is much simpler:

Compute the best value for every position using information from previously solved positions.

Whenever you recognize this pattern, you should immediately think: **"This is a 1-D Dynamic Programming problem."**

Bootcamp Summary

What We Learned

In this problem, we learned how to maximize the amount of treasure collected while moving through a sequence of checkpoints with restricted jump lengths. Although the story is about a treasure caravan, the underlying concept is a classic **1-D Dynamic Programming** problem. The key idea is to compute the **best possible answer for every checkpoint** and reuse those results instead of exploring every possible path.

Core DP State

We defined `dp[i]` as

The maximum treasure that can be collected upon reaching checkpoint i .

Instead of remembering every possible path, we only store the **best** one. This dramatically reduces the amount of computation.

State Transition

From checkpoint i we can move to

$i + 1$

$i + 2$

$i + K$

So whenever $dp[i]$ is known, we update

$dp[i+1]$

$dp[i+2]$

$dp[i+K]$

using

```
max(current_value,  
new_value)
```

to keep only the best answer.

Initialization

The caravan always starts at Checkpoint 1 Therefore,

$dp[1]$

=

$A[1]$

Every other checkpoint starts as $-\infty$ because it has not been reached yet.

Processing Order

Since every move goes only

Left

↓

Right

there are no cycles. Processing checkpoints from $1 \rightarrow N$ automatically guarantees that all previous information is already available before updating future checkpoints.

Why Dynamic Programming?

A brute-force solution explores every possible path. With three choices at each checkpoint, the number of paths grows exponentially. $\approx 3^N$ which is impossible for large values of N . Dynamic Programming computes each checkpoint only once, reducing the complexity to $O(N)$ per test case.

Important Concepts Reinforced

After solving this problem, you should be comfortable with:  Defining a DP state 
Initializing unreachable states  Performing state transitions  Updating states using `max()`  Processing DP iteratively  Avoiding repeated computations  Recognizing DP patterns hidden inside stories

Complexity

Complexity	Value
Time	$O(N)$
Space	$O(N)$

This is the optimal solution because every checkpoint must be processed at least once.

Similar Problems to Practice

If you understood this problem, you should next try:

1. Climbing Stairs
2. Frog Jump
3. Minimum Cost Climbing Stairs
4. House Robber
5. Coin Change
6. Longest Increasing Subsequence (LIS)

These problems strengthen the same Dynamic Programming intuition from different perspectives.

Final Takeaway

Whenever a problem asks you to find the **best answer at each position**, and the current answer depends on a few previously computed positions, think about **Dynamic Programming**. The general workflow is:

Define the State

↓

Find the Transition

↓

Initialize the Base Case

↓

Process States in Order

↓

The Final State Contains the Answer

Mastering this pattern will help you solve many more Dynamic Programming problems in future contests.

Problem 6 — The Number Pyramid

Problem Statement

Deep inside the Pyramid of Numbers, there is a magical staircase. Legend says that each step reveals the natural numbers in order, starting from **1**. The first step shows only the number **1**. Each new step adds the next number while keeping all the previous ones. Your task is to recreate this magical staircase. Given an integer **N**, print the first **N** steps of the Number Staircase. Each step **i** should contain all integers from **1** to **i**, separated by a single space.

Input Format

The first and only line contains a single integer **N**.

Constraints

- $1 \leq N < 100$

Output Format

Print the required Number Staircase.

Sample Input

3

Sample Output

```
1
1 2
1 2 3
```

Solution (C)

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);

    for (int i = 1; i ≤ n; i++)
    {
        for (int j = 1; j ≤ i; j++)
        {
            printf("%d", j);

            if (j ≠ i)
                printf(" ");
        }

        printf("\n");
    }

    return 0;
}
```

Thinking Process

At first glance, this looks like a pattern printing problem. The statement asks us to print the first **N** rows. Let's observe a few rows.

N = 5

1

1 2

1 2 3

1 2 3 4

1 2 3 4 5

Notice something interesting. For every row:

- The row number tells us **how many numbers** to print.
- The numbers always start from **1**.
- The numbers increase by **1** each time.
- Every row ends with its own row number.

For example, Row 4 is 1 2 3 4 Row 5 is 1 2 3 4 5 This immediately suggests two loops. The **outer loop** decides which row we are printing.

Row = 1

Row = 2

...

Row = N

For each row, we need another loop. The **inner loop** prints

1

2

3

...

Row

Finally, after finishing one row, we move to the next line. So the algorithm is simply:

1. Loop through every row.
2. Print numbers from **1** to the current row number.
3. Print a newline.

No arrays, no recursion, and no special data structures are required. This is a straightforward nested-loop pattern printing problem.

Problem 7 — Restoring the Oath

Problem Statement

In the ancient city of **Veyrun**, nine towers stand along the Avenue of Houses, one for each founding family, built in the order their oaths were sworn. Tradition demands that each tower stand **strictly taller** than the one before it, a visible record that every generation built higher than the last. But an earthquake has struck. Some towers sank, others didn't move at all, and the ancient order has been broken — a tower may now stand **shorter than, or equal to**, the one before it. You are the city's mason. You may raise any tower's height by **one brick** with a single unit of labor. Towers can **only be built up**, never torn down. Restore the order so that every tower is **strictly taller** than

the tower before it using the **minimum total number of bricks**. Given the current heights of the **N** towers, find the minimum number of increment operations required.

Input Format

- The first line contains an integer **N** — the number of elements in the array.
- The second line contains **N** space-separated integers **A₁, A₂, ..., A_n**.

Constraints

- $1 \leq N \leq 3 \times 10^5$
- $-10^9 \leq A_i \leq 10^9$

Output Format

Print a single integer — the minimum number of increment operations required.

Sample Input 0

```
7
1 1 1 1 1 1 1
```

Sample Output 0

```
21
```

Explanation 0

The array must become at least:

1 2 3 4 5 6 7

Total increments:

$$0 + 1 + 2 + 3 + 4 + 5 + 6 = 21$$

Sample Input 1

5
4 1 3 2 5

Sample Output 1

15

Explanation 1

The array becomes:

4 5 6 7 8

Total increments:

$$0 + 4 + 3 + 5 + 3 = 15$$

Solution (C)

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);

    long long operations = 0;
    long long previous;

    scanf("%lld", &previous);

    for (int i = 1; i < n; i++)
    {
        long long current;
        scanf("%lld", &current);

        if (current ≤ previous)
        {
            operations += (previous + 1 - current);
            current = previous + 1;
        }

        previous = current;
    }

    printf("%lld\n", operations);

    return 0;
}
```

Thinking Process

The array must become **strictly increasing**. This means that for every position i ($i > 1$), $A[i] > A[i - 1]$. Since we are **only allowed to increase** values, we process the array from **left to right**. For each element, there are only two possibilities.

Case 1: Current element is already valid

If $A[i] > A[i - 1]$ then no operation is needed. Example: 4 7 Since $7 > 4$ we keep it unchanged.

Case 2: Current element is too small

If $A[i] \leq A[i - 1]$ then we must increase it until it becomes $A[i - 1] + 1$. The number of operations required is $(A[i - 1] + 1) - A[i]$. After increasing it, this new value becomes the previous element for the next iteration.

Because each element depends only on the previous one, we can solve the problem in a **single left-to-right traversal**. No sorting or extra arrays are needed. The total answer is simply the sum of all increments performed.

Dry Run

Consider the following input:

```
5
4 1 3 2 5
```

Initial array:

4 1 3 2 5

Total operations = **0**

Step 1

Current array:

4 1 3 2 5

Compare: $1 \leq 4$ So, increase **1** to **5**. Operations added: $5 - 1 = 4$ Array becomes:

4 5 3 2 5

Total operations = **4**

Step 2

Compare: $3 \leq 5$ Increase **3** to **6**. Operations added: $6 - 3 = 3$ Array becomes:

4 5 6 2 5

Total operations = **7**

Step 3

Compare: $2 \leq 6$ Increase **2** to **7**. Operations added: $7 - 2 = 5$ Array becomes:

4 5 6 7 5

Total operations = **12**

Step 4

Compare: $5 \leq 7$ Increase **5** to **8**. Operations added: $8 - 5 = 3$ Final array:

4 5 6 7 8

Total operations:

$4 + 3 + 5 + 3 = 15$

Output:

15

Complexity Analysis

Let **N** be the number of elements in the array.

Time Complexity

We traverse the array exactly once. Therefore, Time Complexity = $O(N)$

Space Complexity

We only use a few variables (`previous` , `current` , and `operations`) regardless of the input size. No extra arrays or data structures are required. Therefore,

Space Complexity = $O(1)$

Common Mistakes

1. Making the current element equal to the previous element

Wrong:

```
current = previous
```

This produces:

```
4 5 5 6
```

which is **not strictly increasing**. It must be:

```
current = previous + 1
```

2. Forgetting to update the current value

Wrong:

```
operations += previous + 1 - current;
```

but forgetting

```
current = previous + 1;
```

The next comparison will use the old value and produce an incorrect answer.

3. Using `int` for the answer

The number of operations can become very large. For example, $N = 300000$. The total increments may exceed the range of a 32-bit integer. Always use:

```
long long operations;
```

4. Starting the loop from index 0

The first element has no previous element. Comparisons should begin from the **second element**.

5. Sorting the array

Some beginners try:

```
Sort → Make increasing
```

This changes the original order of the towers, which is **not allowed**. The order of the array must remain unchanged.

Pattern Recognition

Key Observation

Each element only needs to satisfy one condition: $A[i] > A[i - 1]$. If it already does, leave it unchanged. Otherwise, increase it to the **smallest possible valid value**:

$A[i] = A[i - 1] + 1$. Choosing any larger value would only increase the total operations unnecessarily.

Pattern

This is a **Greedy** problem. At every position, we make the **locally optimal** decision:

- Do nothing if the current value is already valid.
- Otherwise, increase it by the minimum amount required.

Since every future element depends only on the updated previous element, this greedy choice is always optimal.

How to Identify This Pattern

When a problem contains phrases like:

- **Minimum increments**
- **Only increase values**
- **Maintain original order**
- **Strictly increasing**

- **Minimum operations**

it is often a hint that a **left-to-right greedy traversal** is sufficient.

Summary

- Traverse the array from **left to right**.
- Keep track of the **previous (updated) element**.
- If the current element is already greater than the previous one, leave it unchanged.
- Otherwise, increase it to **previous + 1**.
- Add the number of increments to the answer.
- Continue until the end of the array.

Key Formula

If $A[i] \leq A[i - 1]$ then

```
operations += (A[i - 1] + 1) - A[i]
A[i] = A[i - 1] + 1
```

Complexity

Time : $O(N)$

Space : $O(1)$

Pattern

Greedy

Problem 8 — Harmonics of the Lexicon

Problem Statement

A choir of **M** strings, each consisting only of lowercase English letters, is given to you — one string per line. For any string **s**, define its signature vector

$f(s) = (f_1(s), f_2(s), \dots, f_{26}(s))$ where **f_c(s)** is the number of times the **c-th** letter of the alphabet occurs in **s**. For two strings **s_i** and **s_j**, define their **resonance** as

$$R(s_i, s_j) = \sum_{c=1..26} (f_c(s_i) \times f_c(s_j))$$

Your task is to compute the total resonance across every **unordered pair** of strings in the choir:

$$\text{Answer} = \sum_{1 \leq i < j \leq M} R(s_i, s_j)$$

Since this value can be very large, output it modulo **10⁹ + 7**.

Function Description

Complete the function **sumOfResonance** in the editor below.

`sumOfResonance(string s[M])`

Returns

- **int** — the value

$$(\sum R(s_i, s_j)) \bmod (10^9 + 7)$$

Input Format

The first line contains a single integer **T** — the number of test cases. For each test case:

- The first line contains a single integer **M** — the number of strings.
- Each of the next **M** lines contains one string.

Constraints

- $1 \leq T \leq 20$
- $1 \leq M \leq 10^5$
- $1 \leq |s_i| \leq 10^5$
- The sum of the lengths of all strings within a test case does not exceed 2×10^5 .
- Every string consists only of lowercase English letters (a – z).

Output Format

For each test case, output a single integer — the total resonance across all unordered pairs, modulo $10^9 + 7$.

Sample Input 0

```
1
3
aab
abc
bcc
```

Sample Output 0

```
7
```

Explanation 0

The signature vectors (restricted to letters a, b, c, since no other letters appear) are

$f(aab) = (2, 1, 0)$, $f(abc) = (1, 1, 1)$, $f(bcc) = (0, 1, 2)$. Computing

resonance for each pair: $R(aab, abc) = 2 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 = 3$,

$R(aab, bcc) = 2 \cdot 0 + 1 \cdot 1 + 0 \cdot 2 = 1$, $R(abc, bcc) = 1 \cdot 0 + 1 \cdot 1 + 1 \cdot 2 = 3$. Total
 $= 3 + 1 + 3 = 7$.

Sample Input 1

```
3
3
aab
abc
bcc
1
zzzzz
4
abc
abc
abc
abc
```

Sample Output 1

```
7
0
18
```

Explanation 1

Scenario 1 — Same as the original sample: $M = 3$ strings `aab` , `abc` , `bcc` , pairwise resonances summing to `7` .

Scenario 2 — $M = 1$, just `zzzzz`. With only one string, there are no pairs (i, j) with $i < j$, so the sum is 0 .

Scenario 3 — $M = 4$ identical strings, all `abc`. Every string has signature vector $f = (1, 1, 1, 0, \dots, 0)$, so every pair resonates identically: $R(s_i, s_j) = 3$. There are $\binom{4}{2} = 6$ pairs, giving total $6 \times 3 = 18$.

Solution (C)

```
#include <stdio.h>
#include <string.h>

#define MAXM 100000
#define MOD 1000000007LL

int freq[26];
char s[100005];

int main()
{
    int T;
    scanf("%d", &T);

    while (T--)
    {
        int M;
        scanf("%d", &M);

        long long answer = 0;

        memset(freq, 0, sizeof(freq));

        for (int i = 0; i < M; i++)
        {
            scanf("%s", s);

            int cnt[26] = {0};

            int len = strlen(s);

            for (int j = 0; j < len; j++)
                cnt[s[j] - 'a']++;

            for (int c = 0; c < 26; c++)
            {
                answer = (answer + 1LL * cnt[c] * freq[c]) % MOD;
            }
        }
    }
}
```

```

        freq[c] += cnt[c];
    }
}

printf("%lld\n", answer);
}

return 0;
}

```

Thinking Process

At first, the problem seems to ask us to compare every pair of strings. For each pair, we could count how many times each of the 26 letters appears in both strings and compute $R(s_i, s_j) = \sum f_c(s_i) \times f_c(s_j)$. However, this approach is far too slow. If there are **M** strings, comparing every pair requires $O(M^2)$ operations, which is impossible for $M \leq 100000$.

A Better Observation

Notice that the resonance is simply the sum of the contributions of each letter independently. For example,

$$\begin{aligned}
 R(s_i, s_j) &= \text{contribution of 'a'} \\
 &+ \text{contribution of 'b'} \\
 &+ \dots \\
 &+ \text{contribution of 'z'}
 \end{aligned}$$

So instead of comparing whole strings, we can process one letter at a time.

Process Strings One by One

Suppose we are currently reading a new string. For each letter:

- Count how many times it appears in the current string.
- Also remember how many times this letter has appeared in **all previous strings**.

For example, Current string: aabcc Frequency:

$$a = 2$$

$$b = 1$$

$$c = 2$$

Suppose previous strings have contributed

$$a = 7$$

$$b = 3$$

$$c = 5$$

Then this string contributes

$$2 \times 7$$

+

$$1 \times 3$$

+

$$2 \times 5$$

to the final answer. After adding this contribution, we update the stored frequencies by adding the current string's frequencies.

Why This Works

Every unordered pair (previous string, current string) is counted exactly once. When processing the current string, all previous strings have already been included in the accumulated frequencies, while future strings have not been processed yet. Thus every valid pair contributes exactly once to the answer.

Instead of comparing every pair of strings, we only maintain cumulative letter frequencies, reducing the solution to a single pass over the input.

Dry Run

Consider one test case:

`M = 3`

`aab`

`abc`

`bcc`

Initially,

`freq[a..z] = 0`

`answer = 0`

Process String 1

`aab` Character frequencies:

$$a = 2$$

$$b = 1$$

$$c = 0$$

Contribution:

$$2 \times 0 + 1 \times 0 = 0$$

Update cumulative frequencies:

$$a = 2$$

$$b = 1$$

$$c = 0$$

Current answer:

$$0$$

Process String 2

abc Character frequencies:

$$a = 1$$

$$b = 1$$

$$c = 1$$

Contribution:

$$1 \times 2$$

+

$$1 \times 1$$

+

$$1 \times 0$$

$$= 3$$

Update cumulative frequencies:

$$a = 3$$

$$b = 2$$

$$c = 1$$

Current answer:

$$3$$

Process String 3

bcc Character frequencies:

$$a = 0$$

$$b = 1$$

$$c = 2$$

Contribution:

$$0 \times 3$$

+

$$1 \times 2$$

+

$$2 \times 1$$

$$= 4$$

Update cumulative frequencies:

$$a = 3$$

$$b = 3$$

$$c = 3$$

Final answer:

$$3 + 4 = 7$$

Output:

7

Complexity Analysis

Let:

- **M** = number of strings
- **L** = total length of all strings in a test case

Time Complexity

For every string:

1. Count the frequency of its characters.
2. Iterate over the 26 lowercase English letters.

Counting character frequencies takes $O(\text{length of string})$ and processing the 26 letters takes $O(26)$. Since 26 is a constant, the total complexity is $O(L + 26 \times M)$ which simplifies to $O(L)$.

Space Complexity

We use:

- An array of size **26** to store cumulative frequencies.
- Another array of size **26** for the current string.

Both are constant-sized arrays. Therefore, Space Complexity = $O(1)$ (ignoring the input string itself).

Common Mistakes

1. Comparing Every Pair of Strings

A common first idea is:

```
for every i
  for every j > i
    compute R(si, sj)
```

This requires

$O(M^2)$

pairwise comparisons, which is far too slow for

 $M \leq 100000$

2. Forgetting That Each Letter Contributes Independently

Some solutions try to compare whole strings character by character. Instead, compute the contribution of each letter separately:

```
answer += current_frequency × previous_total_frequency
```

for all 26 letters.

3. Updating the Cumulative Frequency Before Calculating the Contribution

Wrong order:

```
freq[c] += cnt[c];
answer += cnt[c] * freq[c];
```

This incorrectly counts the current string with itself. The correct order is:

```
answer += 1LL * cnt[c] * freq[c];
freq[c] += cnt[c];
```

4. Integer Overflow

The product

```
cnt[c] * freq[c]
```

can exceed the range of a 32-bit integer. Always write:

```
1LL * cnt[c] * freq[c]
```

and store the answer in a

```
long long
```

variable.

5. Forgetting to Take Modulo

The answer can become extremely large. Always update it as

```
answer = (answer + value) % MOD;
```

instead of waiting until the very end.

Pattern Recognition

Key Observation

The resonance between two strings is the sum of the contributions of each letter:

$$R(s_i, s_j) = \sum (\text{frequency of letter } c \text{ in } s_i) \times (\text{frequency of letter } c \text{ in } s_j)$$

Since each letter contributes independently, we can process the strings one by one while maintaining the cumulative frequency of every letter seen so far.

Pattern

This problem is an example of **Frequency Accumulation**. Instead of comparing every pair of strings, we keep track of the total occurrences of each character in all previously processed strings. When a new string arrives:

1. Count the frequency of each letter.
2. Compute its contribution using the previously accumulated frequencies.
3. Add its frequencies to the cumulative totals.

This ensures every unordered pair is counted exactly once.

How to Identify This Pattern

When a problem involves:

- Pairwise contribution
- Dot product / frequency product
- Character frequencies
- Summation over all pairs
- Very large input size where $O(N^2)$ is impossible

look for a way to **accumulate information from previously processed items** instead of comparing every pair directly. This often leads to a **Frequency Accumulation** or **Prefix Contribution** solution.

Summary

- Process the strings **one by one**.
- For each string, count the frequency of all **26 lowercase letters**.
- For every letter, compute its contribution using the cumulative frequency from all previously processed strings.
- Add this contribution to the answer.
- Update the cumulative frequencies with the current string's frequencies.
- Repeat for all strings and output the answer modulo $10^9 + 7$.

Key Formula

For each character c :

```
answer += currentFreq[c] × totalPreviousFreq[c]
```

Then update:

```
totalPreviousFreq[c] += currentFreq[c]
```


Complexity

Time : $O(L)$

Space : $O(1)$

where **L** is the total length of all strings in a test case.

Pattern

Frequency Accumulation

Prefix Contribution

Problem 9 — The Guildmaster's Selection

The guildmaster of an ancient order must assemble a team by choosing **r** apprentices out of a pool of **n** candidates, where the order of selection does **not** matter. For each test case, determine the number of distinct ways the guildmaster can form the team, i.e., compute the binomial coefficient:

$$C(n, r) = \frac{n!}{r!(n-r)!}$$

Since the answer can be extremely large, print it modulo:

1000000007

Function Description

Complete the function:

```
countSelections(n, r)
```

Parameters:

- n — number of candidates
- r — number of apprentices to choose

Returns:

- $C(n, r) \bmod 1000000007$

Input Format

- The first line contains an integer T — the number of test cases.
- Each of the next T lines contains two integers:

```
n r
```

Constraints

$$1 \leq T \leq 100000$$

$$0 \leq r \leq n \leq 1000000$$

Output Format

For each test case, print:

$C(n,r) \bmod 1000000007$

Sample Input 0

```
3
5 2
6 0
10 10
```

Sample Output 0

```
10
1
1
```

Explanation 0

$C(5,2) = 10$
 $C(6,0) = 1$
 $C(10,10) = 1$

Sample Input 1

```
3
0 0
7 3
1000000 500000
```

Sample Output 1

```
1
35
996692777
```

Solution

```

#include <stdio.h>

#define MAX 1000000
#define MOD 1000000007LL

long long fact[MAX + 1];
long long invFact[MAX + 1];

long long power(long long a, long long b)
{
    long long result = 1;

    while (b > 0)
    {
        if (b & 1)
            result = (result * a) % MOD;

        a = (a * a) % MOD;
        b >>= 1;
    }

    return result;
}

void precompute()
{
    fact[0] = 1;

    for (int i = 1; i ≤ MAX; i++)
        fact[i] = (fact[i - 1] * i) % MOD;

    invFact[MAX] = power(fact[MAX], MOD - 2);

    for (int i = MAX; i ≥ 1; i--)
        invFact[i - 1] = (invFact[i] * i) % MOD;
}

```

```

long long nCr(int n, int r)
{
    if (r < 0 || r > n)
        return 0;

    return (((fact[n] * invFact[r]) % MOD) * invFact[n - r]) % MOD;
}

int main()
{
    precompute();

    int T;
    scanf("%d", &T);

    while (T--)
    {
        int n, r;
        scanf("%d %d", &n, &r);

        printf("%lld\n", nCr(n, r));
    }

    return 0;
}

```

Thinking Process

A direct implementation of the formula

$$C(n, r) = \frac{n!}{r!(n-r)!}$$

is impossible for the given constraints. Why?

- n can be as large as 10^6 .
- There can be up to 10^5 test cases.
- Computing factorial separately for every query would take

$O(n)$

per test case, resulting in

$O(T \times n)$

which is far too slow.

Observation 1

The modulus

$MOD = 1000000007$

is a **prime number**. This allows us to perform division in modular arithmetic using the modular inverse. Instead of

a / b

we compute

$a \times \text{inverse}(b)$

where

$$\text{inverse}(b) = b^{(\text{MOD}-2)} \bmod \text{MOD}$$

using **Fermat's Little Theorem**.

Observation 2

Many test cases share the same range of values. Instead of recomputing factorials repeatedly, precompute them once. Create an array:

$$\text{fact}[i] = i! \bmod \text{MOD}$$

for every

$$0 \leq i \leq 10^6$$

This preprocessing takes only

$$O(\text{MAX})$$

time.

Observation 3

Similarly, precompute the inverse factorials:

$$\text{invFact}[i] = (i!)^{-1} \bmod \text{MOD}$$

Once both arrays are ready, the combination formula becomes

$$\begin{aligned} C(n, r) &= \\ &= \text{fact}[n] \\ &\times \text{invFact}[r] \\ &\times \text{invFact}[n-r] \\ &(\bmod \text{MOD}) \end{aligned}$$

Each query now requires only a few multiplications.

Final Idea

1. Precompute all factorials.
2. Compute the modular inverse of the largest factorial using fast exponentiation.
3. Generate all inverse factorials.
4. Answer every query using the precomputed arrays.

This reduces each test case from **$O(n)$** to **$O(1)$** after preprocessing.

Dry Run

Consider one test case:

$$\begin{aligned} n &= 5 \\ r &= 2 \end{aligned}$$

We need to compute:

$$C(5, 2)$$

Step 1: Precompute Factorials

```
fact[0] = 1
fact[1] = 1
fact[2] = 2
fact[3] = 6
fact[4] = 24
fact[5] = 120
```

(All values are stored modulo 1000000007 .)

Step 2: Precompute Inverse Factorials

Using Fermat's Little Theorem,

```
invFact[5] = inverse(120)
```

Then compute backwards:

```
invFact[4]
invFact[3]
invFact[2]
invFact[1]
invFact[0]
```

Step 3: Apply the Formula

```
C(5,2)
=
fact[5]
× invFact[2]
× invFact[3]
(mod MOD)
```

Substituting the actual factorial values:

```
=
120
× inverse(2)
× inverse(6)
```

Since

```
inverse(2) = 1/2
inverse(6) = 1/6
```

the result becomes

$$120 \times 1/2 \times 1/6 \\ = 10$$

After taking modulo:

$$10$$

Another Example

$$n = 10 \\ r = 10$$

Formula:

$$C(10,10) \\ = \\ \text{fact}[10] \\ \times \text{invFact}[10] \\ \times \text{invFact}[0]$$

Since

$$\text{fact}[10] \times \text{invFact}[10] = 1 \\ \text{fact}[0] = 1$$

Answer:

1

Boundary Example

`n = 6`

`r = 0`

Formula:

`C(6,0)`

`=`

`fact[6]`

`× invFact[0]`

`× invFact[6]`

Again,

`fact[6] × invFact[6] = 1`

Therefore,

`Answer = 1`

Complexity Analysis

Let:

- **MAX = 10^6** (maximum possible value of n)
- **T** = number of test cases

Precomputation

We compute:

- All factorials from 0 to MAX
- All inverse factorials from MAX back to 0

Each array is computed only once. Time required:

$$O(MAX)$$

Fast exponentiation is used once to compute:

$$fact[MAX]^{(MOD-2)}$$

which takes

$$O(\log MOD)$$

Since $\log MOD$ is much smaller than MAX , the total preprocessing time is

$$O(MAX)$$

Per Test Case

Each query computes

```
fact[n]
× invFact[r]
× invFact[n-r]
```

using only a few modular multiplications. Therefore,

Time Complexity = $O(1)$

per query.

Overall Time Complexity

$O(\text{MAX} + T)$

where:

- $O(\text{MAX})$ for preprocessing
- $O(1)$ for each of the T test cases

Space Complexity

We store two arrays:

```
fact[MAX + 1]  
invFact[MAX + 1]
```

Each contains $\text{MAX} + 1$ elements. Therefore,

Space Complexity = $O(\text{MAX})$

Common Mistakes

1. Computing Factorials for Every Test Case

A common approach is:

```
For every query:  
    Compute n!  
    Compute r!  
    Compute (n-r)!
```

This requires

$O(n)$

time per query. With

$T = 100000$

this solution becomes far too slow. Instead, precompute factorials only once.

2. Performing Normal Division Under Modulo

Many beginners write:

$$\text{fact}[n] / (\text{fact}[r] * \text{fact}[n-r])$$

This is incorrect in modular arithmetic. Instead, use the modular inverse:

$$a / b$$

↓

$$a * \text{inverse}(b)$$

where

$$\text{inverse}(b) = b^{(\text{MOD}-2)} \bmod \text{MOD}$$

because MOD is prime.

3. Forgetting to Precompute Inverse Factorials

Some solutions compute

$$\text{power}(x, \text{MOD}-2)$$

for every query. Although correct, this adds an unnecessary

$$O(\log \text{MOD})$$

cost per test case. Precomputing all inverse factorials makes each query run in

$O(1)$

time.

4. Not Handling Boundary Cases

These cases must always return:

$$C(n, 0) = 1$$

$$C(n, n) = 1$$

$$C(0, 0) = 1$$

Your implementation should naturally produce these values.

5. Integer Overflow

Factorials become extremely large. Always perform multiplication like this:

```
(result * value) % MOD
```

and use

```
long long
```

instead of `int` for intermediate calculations.

Pattern Recognition

Key Observation

The problem asks us to compute

$$C(n, r)$$

for a very large number of test cases. Computing factorials repeatedly would be too slow. Since all queries use the same modulus and the maximum value of n is known in advance, we can preprocess the required values once and reuse them.

Pattern

This problem is a classic example of **Precomputation**. We build two arrays:

$$\begin{aligned} \text{fact}[i] &= i! \bmod \text{MOD} \\ \text{invFact}[i] &= (i!)^{-1} \bmod \text{MOD} \end{aligned}$$

After preprocessing, every query can be answered in constant time using:

$$\begin{aligned} C(n, r) &= \\ &= \text{fact}[n] \\ &\times \text{invFact}[r] \\ &\times \text{invFact}[n-r] \\ &(\bmod \text{MOD}) \end{aligned}$$

Supporting Technique

The solution also uses **Modular Arithmetic**. Because

$$\text{MOD} = 1000000007$$

is prime, Fermat's Little Theorem gives

$$a^{-1}$$

=

$$a^{(\text{MOD}-2)}$$

$$(\text{mod } \text{MOD})$$

This allows division under modulo by converting it into multiplication with a modular inverse.

How to Identify This Pattern

When a problem has:

- Many independent queries
- Same mathematical formula for every query
- Large constraints
- A fixed maximum value
- A prime modulus

think about:

Precomputation

+

Modular Arithmetic

+

Fast Exponentiation

These techniques often reduce a slow **$O(n)$** solution per query to **$O(1)$** after one-time preprocessing.

Summary

- The answer is the binomial coefficient:

$$C(n, r) = \frac{n!}{r!(n-r)!}$$

- Direct computation is too slow because there can be up to **100000** queries and n can be as large as **10^6** .
- Precompute:
 - `fact[i] = i! mod MOD`
 - `invFact[i] = (i!)-1 mod MOD`
- Compute the modular inverse using **Fermat's Little Theorem** and **Fast Exponentiation**.
- Answer each query using:

$C(n, r)$

=

`fact[n]`

`× invFact[r]`

`× invFact[n-r]`

`(mod MOD)`

Key Formula

$C(n, r)$

=

`fact[n]`

`× invFact[r]`

`× invFact[n-r]`

`(mod 1000000007)`

where

`inverse(a)`

=

`a^(MOD-2) mod MOD`

Complexity

Preprocessing : $O(\text{MAX})$

Each Query : $O(1)$

Overall : $O(\text{MAX} + T)$

Space : $O(\text{MAX})$

where **MAX** = 10^6 .

Pattern

```
Precomputation
+
Modular Arithmetic
+
Fast Exponentiation
+
Modular Inverse (Fermat's Little Theorem)
```

Problem 10 — Paths of Least Peril

A trade guild operates across **n** cities, connected by **m** bidirectional roads. Each road connects two cities and has a **risk level**. For a path between two cities, define its **peril** as the **maximum risk level** among all roads used on that path.

$\text{peril}(P) = \text{maximum edge weight on } P$

For each query (a, b) , determine the **minimum possible peril** among all paths from city a to city b . In other words,

`answer(a,b)`

=

minimum possible value of

maximum edge weight

along any path from a to b .

If no path exists between the two cities, print:

-1

Function Description

Complete the function:

`minimizeMaxRisk(...)`

Parameters:

- n — number of cities
- $u[], v[], w[]$ — endpoints and risk level of each road
- $a[], b[]$ — query endpoints

Returns:

`ans[i]`

where

`ans[i]`

=

minimum possible peril

or

-1

if the cities are disconnected.

Input Format

T

For each test case:

n m

Next m lines:

u v w

Then

q

Next q lines:

a b

Constraints

$$1 \leq T \leq 10$$

$$1 \leq n \leq 100000$$

$$0 \leq m \leq 200000$$

$$1 \leq q \leq 100000$$

$$1 \leq w \leq 10^9$$

The graph:

- may be disconnected
- may contain multiple edges between the same pair of cities

Output Format

For every query, output

answer(a,b)

or

-1

if no path exists.

Sample Input 0

```
1
6 5
4 5 1
2 3 2
1 3 3
1 2 4
3 4 5
3
1 5
2 4
1 6
```

Sample Output 0

```
5
5
-1
```

Explanation 0

Cities 1–5 are connected by the 5 given roads; city 6 is isolated. Sorting roads by risk and running Kruskal's algorithm builds the MST: $(4, 5, 1)$, $(2, 3, 2)$, $(1, 3, 3)$, $(3, 4, 5)$ — road $(1, 2, 4)$ is skipped since 1 and 2 are already connected once $(1, 3, 3)$ and $(2, 3, 2)$ are added. This gives a tree where city 3 is the hub connecting to 1, 2, and 4, with 4 further connected to 5.

- Query (1, 5) : the tree path is $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$, risk levels 3, 5, 1 , so peril
= $\max(3,5,1) = 5$.
- Query (2, 4) : the tree path is $2 \rightarrow 3 \rightarrow 4$, risk levels 2, 5 , so peril
= $\max(2,5) = 5$.
- Query (1, 6) : city 6 has no roads at all, so it is disconnected from city 1. Answer
= -1 .

Sample Input 1

```

2
6 5
4 5 1
2 3 2
1 3 3
1 2 4
3 4 5
3
1 5
2 4
1 6
4 4
1 2 2
2 3 1
1 3 100
2 3 1
3
1 3
2 1
1 4

```

Sample Output 1

```
5
5
-1
2
2
-1
```

Explanation 1

Case 1 is the original sample above, giving 5, 5, -1 .

Case 2 tests parallel edges: cities 1 and 2 are connected by two roads (risk 2 and risk 10), plus roads (1,3,100) and (2,3,1) , with city 4 isolated. Sorting by weight and running Kruskal gives (2,3,1) and (1,2,2) in the MST — the heavier duplicate (1,2,10) and (1,3,100) are correctly skipped since 1, 2, 3 are already connected. This checks that an implementation compares parallel edges by weight rather than blindly using the first one seen between a pair. With this MST, the path $1 \rightarrow 2 \rightarrow 3$ gives query (1,3) peril $\max(2,1) = 2$; query (2,1) uses the direct MST edge, peril 2 ; query (1,4) hits the isolated node, giving -1 .

Solution

```

#include <stdio.h>
#include <stdlib.h>

#define MAXN 100005
#define MAXM 200005
#define LOG 17

typedef struct {
    int u, v, w;
} Edge;

typedef struct {
    int to, w, next;
} Node;

Edge edges[MAXM];
Node graph[2 * MAXN];
int head[MAXN];
int parent[MAXN];
int depth[MAXN];
int up[MAXN][LOG + 1];
int mx[MAXN][LOG + 1];
int component[MAXN];

int edgeCnt;

int cmp(const void *a, const void *b) {
    return ((Edge *)a)→w - ((Edge *)b)→w;
}

int find(int x) {
    if (parent[x] == x)
        return x;
    return parent[x] = find(parent[x]);
}

void unite(int a, int b) {

```

```

    a = find(a);
    b = find(b);

    if (a  $\neq$  b)
        parent[a] = b;
}

void addEdge(int u, int v, int w) {
    graph[++edgeCnt].to = v;
    graph[edgeCnt].w = w;
    graph[edgeCnt].next = head[u];
    head[u] = edgeCnt;

    graph[++edgeCnt].to = u;
    graph[edgeCnt].w = w;
    graph[edgeCnt].next = head[v];
    head[v] = edgeCnt;
}

void dfs(int u, int p, int wt, int comp) {
    component[u] = comp;
    up[u][0] = p;
    mx[u][0] = wt;

    for (int i = 1; i  $\leq$  LOG; i++) {
        up[u][i] = up[up[u][i - 1]][i - 1];
        mx[u][i] =
            mx[u][i - 1] > mx[up[u][i - 1]][i - 1]
                ? mx[u][i - 1]
                : mx[up[u][i - 1]][i - 1];
    }

    for (int e = head[u]; e; e = graph[e].next) {
        int v = graph[e].to;

        if (v == p)
            continue;

        depth[v] = depth[u] + 1;
        dfs(v, u, graph[e].w, comp);
    }
}

```

```

    }
}

int query(int u, int v) {
    if (component[u] != component[v])
        return -1;

    int ans = 0;

    if (depth[u] < depth[v]) {
        int t = u;
        u = v;
        v = t;
    }

    for (int i = LOG; i ≥ 0; i--) {
        if (depth[u] - (1 << i) ≥ depth[v]) {
            if (mx[u][i] > ans)
                ans = mx[u][i];
            u = up[u][i];
        }
    }

    if (u == v)
        return ans;

    for (int i = LOG; i ≥ 0; i--) {
        if (up[u][i] != up[v][i]) {
            if (mx[u][i] > ans)
                ans = mx[u][i];

            if (mx[v][i] > ans)
                ans = mx[v][i];

            u = up[u][i];
            v = up[v][i];
        }
    }

    if (mx[u][0] > ans)

```



```

    ans = mx[u][0];

    if (mx[v][0] > ans)
        ans = mx[v][0];

    return ans;
}

int main() {
    int T;
    scanf("%d", &T);

    while (T--) {
        int n, m;
        scanf("%d %d", &n, &m);

        edgeCnt = 0;

        for (int i = 1; i ≤ n; i++) {
            head[i] = 0;
            parent[i] = i;
            depth[i] = 0;
            component[i] = 0;
        }

        for (int i = 0; i < m; i++)
            scanf("%d %d %d",
                &edges[i].u,
                &edges[i].v,
                &edges[i].w);

        qsort(edges, m, sizeof(Edge), cmp);

        for (int i = 0; i < m; i++) {
            int u = edges[i].u;
            int v = edges[i].v;

            if (find(u) ≠ find(v)) {
                unite(u, v);
                addEdge(u, v, edges[i].w);
            }
        }
    }
}

```

```

    }
}

int comp = 0;

for (int i = 1; i ≤ n; i++) {
    if (!component[i]) {
        comp++;
        dfs(i, 0, 0, comp);
    }
}

int q;
scanf("%d", &q);

while (q--) {
    int a, b;
    scanf("%d %d", &a, &b);

    printf("%d\n", query(a, b));
}

return 0;
}

```

Thinking Process

Step 1: Understand What We Need

For every query (a, b) , we must find a path whose **largest edge weight is as small as possible**. Notice that the problem **does not ask for the shortest path**. Instead, it asks us to minimize

Maximum Edge Weight

along the path.

Step 2: Brute Force?

For every query we could:

- Try every possible path.
- Compute the maximum edge weight on each path.
- Pick the minimum among them.

Clearly impossible. The graph can have

$n = 100000$

$m = 200000$

$q = 100000$

Brute force would take far too long.

Step 3: Important Observation

Suppose there are many different paths between two cities. Some paths contain very large edges. Some contain smaller edges. We only care about

the smallest possible maximum edge.

There is a famous graph property:

The path between two nodes in the **Minimum Spanning Tree (MST)** always minimizes the maximum edge weight.

This is called the **Minimum Bottleneck Path Property**. Therefore, instead of searching the original graph, we only need to search the MST.

Step 4: Build the MST

We construct the MST using **Kruskal's Algorithm**. Steps:

1. Sort all edges by weight.
2. Use Disjoint Set Union (Union-Find).
3. Add an edge only if it connects two different components.

The result is a forest (or a tree if the graph is connected).

Step 5: Answer Queries Efficiently

After building the MST, each query becomes:

```
Find the maximum edge weight
on the unique path
between a and b.
```

Since an MST is a tree, there is exactly one path between any two connected nodes.

Step 6: Fast Path Queries

Walking the tree for every query would be too slow. Instead we preprocess the tree using:

- DFS
- Binary Lifting (LCA)

For every node we store:

2^k -th ancestor

and

Maximum edge weight
from the node
to that ancestor.

Then each query is answered in

$O(\log N)$

instead of

$O(N)$

Step 7: Handle Disconnected Graphs

The graph may have multiple connected components. During DFS we assign each node a component number. If

$\text{component}[a] \neq \text{component}[b]$

then

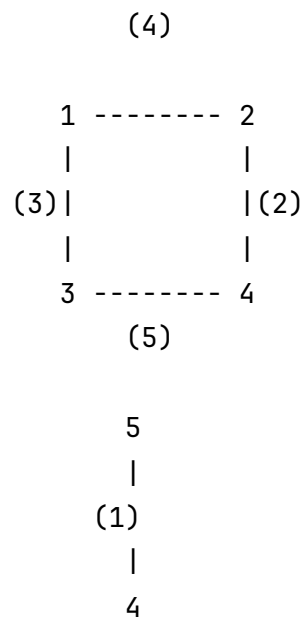
No path exists.

Answer:

-1

Dry Run

Consider the following graph.



Edges:

1 - 3 : 3
2 - 3 : 2
1 - 2 : 4
3 - 4 : 5
4 - 5 : 1

Queries:

(1,5)

(2,4)

(1,6)

Step 1: Sort All Edges

(4,5) = 1

(2,3) = 2

(1,3) = 3

(1,2) = 4

(3,4) = 5

Step 2: Build the MST (Kruskal)

Initially every city is separate.

1 2 3 4 5

Edge (4,5)

Accepted

Tree

4 --1-- 5

Edge (2,3)

Accepted

Tree

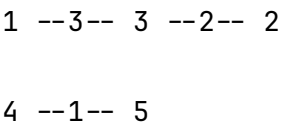
2 --2-- 3

4 --1-- 5

Edge (1,3)

Accepted

Tree



Edge (1,2)

Cities 1 and 2 are already connected.

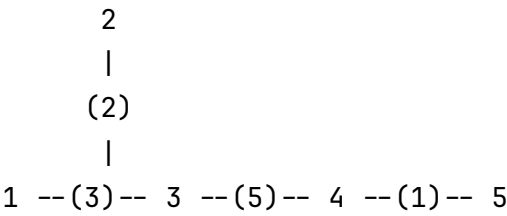
Rejected

because it forms a cycle.

Edge (3,4)

Accepted

Final MST



Query 1

1 → 5

Tree path

1 → 3 → 4 → 5

Edge weights

3

5

1

Maximum edge

$\max(3, 5, 1)$

=

5

Answer

5

Query 2

2 → 4

Tree path

2 → 3 → 4

Edge weights

2

5

Maximum

$\max(2, 5)$

=

5

Answer

5

Query 3

1 → 6

City 6 belongs to another connected component.

No path exists.

Answer

-1

Final Answers

Query (1,5)

↓

5

Query (2,4)

↓

5

Query (1,6)

↓

-1

Complexity Analysis

1. Sorting the Edges

Kruskal's algorithm first sorts all edges. There are m edges. Time:

$O(m \log m)$

2. Building the MST

Each edge performs at most

- one Find
- one Union

Using Disjoint Set Union with Path Compression, each operation is nearly constant.
Time:

$$O(m \alpha(n))$$

where

$$\alpha(n)$$

is the inverse Ackermann function, which is practically constant.

3. DFS Traversal

We visit every node once. Time:

$$O(n)$$

4. Binary Lifting Preprocessing

For every node we compute information for

$$2^0$$

$$2^1$$

$$2^2$$

...

$$2^{\text{LOG}}$$

levels. Time:

$$O(n \log n)$$

5. Processing One Query

Each query climbs the tree using Binary Lifting. At most

$$\text{LOG}$$

jumps are required. Time:

$$O(\log n)$$

6. Processing All Queries

If there are q queries, Time:

$O(q \log n)$

Overall Time Complexity

Sorting MST

+

Building MST

+

DFS

+

Binary Lifting

+

Queries

becomes

$$O(m \log m)$$

+

$$O(n \log n)$$

+

$$O(q \log n)$$

or simply

$$O((m + n + q) \log n)$$

since

$$\log m \approx \log n$$

for this problem.

Space Complexity

We store

- Edge list
- MST adjacency list
- DSU arrays
- Ancestor table
- Maximum-edge table
- Component array

Total memory:

$O(n \log n + m)$

Common Mistakes

1. Using Dijkstra's Algorithm

Many people immediately think of Dijkstra because the graph has weighted edges. However, the problem asks to minimize

Maximum Edge Weight

instead of

Sum of Edge Weights.

These are completely different problems. ❌ Wrong

Shortest Path

✅ Correct

Minimum Bottleneck Path

2. Answering Queries on the Original Graph

After building the MST, some programmers still search the original graph. This is unnecessary. The **Minimum Bottleneck Path Property** guarantees that the answer for every query can be obtained from the MST. Always process queries on the MST.

3. Forgetting Disconnected Components

The graph is **not guaranteed to be connected**. Before answering a query, check whether both cities belong to the same connected component. If not,

```
return -1
```

4. Walking the Tree for Every Query

A simple DFS/BFS for every query takes

 $O(n)$

With up to

100000

queries, this becomes too slow. Use

Binary Lifting (LCA)

to answer each query in

$$O(\log n)$$

5. Storing Only Ancestors

While preprocessing for LCA, some implementations store only

$$\text{up}[\text{node}][k]$$

This is not enough. You must also store

$$\text{mx}[\text{node}][k]$$

which represents the maximum edge weight from the node to its 2^k -th ancestor. Without it, the maximum edge on the path cannot be computed efficiently.

6. Incorrect Maximum Calculation

When lifting both nodes toward their Lowest Common Ancestor (LCA), update the answer every time a jump is made. Correct idea:

$$\text{answer} = \max(\text{answer}, \text{edge used in the jump})$$

If you forget to update the maximum during any jump, the final result will be incorrect.

7. Choosing Edges Without DSU

During Kruskal's algorithm, every edge must be checked using Disjoint Set Union. If cycle detection is skipped, the constructed graph will no longer be an MST, and the bottleneck path property will not hold.

8. Using 32-bit Integers for Edge Weights Incorrectly

Edge weights can be as large as

$$10^9$$

Store weights using at least a 32-bit integer (`int` in C/C++ is sufficient for individual weights). Avoid unnecessary arithmetic that could overflow if edge values are combined.

Pattern Recognition

What Pattern Does This Problem Follow?

This problem is a classic example of the

Minimum Bottleneck Path

pattern. Instead of minimizing the total path cost, we minimize the

Maximum Edge Weight

on the path.

Key Observation

For any two vertices,

The path between them in the MST
has the minimum possible maximum edge.

Therefore,

Original Graph

↓

Build MST

↓

Answer queries on MST

Algorithms Used

1. Kruskal's Algorithm

Purpose:

Build the Minimum Spanning Tree.

Time:

$O(m \log m)$

2. Disjoint Set Union (Union-Find)

Purpose:

Detect cycles while building the MST.

Operations:

Find

Union

Nearly constant time using Path Compression.

3. DFS

Purpose:

Traverse every tree

and compute

- depth
- parent
- connected component

4. Binary Lifting (LCA)

Purpose: Answer queries of the form

Maximum edge
between two nodes

efficiently. Each query takes

$O(\log n)$

instead of

$O(n)$

Overall Pattern

Weighted Graph

↓

Minimum Spanning Tree

↓

Tree

↓

DFS

↓

Binary Lifting (LCA)

↓

Maximum Edge Query

Similar Competitive Programming Problems

If you see questions asking for

- minimum possible maximum edge
- bottleneck path
- minimax path
- path with minimum worst edge
- maximum edge on a tree path
- offline path queries on weighted graphs

then this same pattern is usually applicable:

MST

+

LCA

+

Binary Lifting

Summary

Core Idea

The problem asks us to minimize the **largest edge weight** on a path, **not** the total path weight. A key graph property states:

For any two vertices,
the path between them in the
Minimum Spanning Tree (MST)
minimizes the maximum edge weight.

Therefore, instead of answering queries on the original graph, we:

Original Graph

↓

Build MST

↓

Answer queries on the MST

Complete Approach

Step 1

Sort all edges by weight.

Kruskal's Algorithm

Step 2

Use

Disjoint Set Union (DSU)

to build the MST while avoiding cycles.

Step 3

Run DFS on every connected component to compute:

Depth

Parent

Connected Component

Step 4

Preprocess Binary Lifting tables. For every node store:

$up[node][k]$

and

$mx[node][k]$

where

$mx[node][k]$

is the maximum edge weight from the node to its 2^k -th ancestor.

Step 5

For every query:

- If the two cities are in different connected components:

Answer = -1

- Otherwise:

Raise both nodes to their Lowest Common Ancestor (LCA) while tracking the maximum edge encountered. The final maximum edge is the minimum possible peril.

Time Complexity

Building MST

$O(m \log m)$

DFS + Binary Lifting

$O(n \log n)$

Each Query

$O(\log n)$

Overall:

$O((m + n + q) \log n)$

Space Complexity

$O(n \log n + m)$

Pattern

Minimum Spanning Tree (MST)

↓

Disjoint Set Union (DSU)

↓

Depth First Search (DFS)

↓

Lowest Common Ancestor (LCA)

↓

Binary Lifting

↓

Maximum Edge Query on Tree

Key Takeaway

Whenever a problem asks to find a path that **minimizes the maximum edge weight**, think of the **Minimum Bottleneck Path** property. The standard solution pattern is:

Kruskal's MST

+

DSU

+

Binary Lifting (LCA)

This combination efficiently answers a large number of bottleneck path queries on weighted graphs.

Problem 11 — Hackerman's Network Sweep

Hackerman's network consists of r **routers** connected in a **complete mesh** (every pair of routers is directly connected), and $n - r$ devices. Every device belongs to exactly one router's local cluster. Inside a cluster:

- Devices may connect to their router.
- Devices may connect to other devices in the same cluster.
- Devices **never** connect to another cluster.

However, there is **exactly one rogue edge** that incorrectly connects two different clusters. Your tasks are:

1. Find the rogue edge.
2. Compute the shortest distance from the start node s to every node using the original graph.
3. Remove the rogue edge and compute the shortest distances again.
4. Compute the shortest path from s to t using **only** edges whose bandwidth is at least b_{Min} .

It is guaranteed that removing exactly one edge restores a valid **mesh-and-clusters** topology.

Function Description

Complete the function

```
networkSweep(...)
```

Parameters:

```
n
```

Number of nodes.

```
routers[]
```

The router node labels.

```
edges[][]
```

Each edge is

```
[u, v, latency, bandwidth]
```

```
s
```


Starting node.

`t`

Target node.

`bMin`

Minimum required bandwidth.

Returns

Return a structure containing:

`rogueEdge`

The rogue edge (smaller endpoint first).

`distFull`

Shortest distance from `s` to every node using the original graph.

`distWithoutRogue`

Shortest distance after removing the rogue edge.

`constrainedDist`

Shortest path from s to t using only edges with

$\text{bandwidth} \geq b_{\text{Min}}$

If unreachable,

-1

Input Format

n r

routers[]

m

Next m lines:

u v latency bandwidth

Finally,

s t b_{Min}

Constraints

$$3 \leq r \leq n \leq 2000$$

$$n-1 \leq m \leq 4000$$

$$1 \leq \text{latency} \leq 10000$$

$$1 \leq \text{bandwidth} \leq 10000$$

No multi-edges

The graph is connected. Exactly **one** rogue edge exists and is uniquely identifiable.

Output Format

The driver prints:

Line 1:
rogue edge

Line 2:
distFull

Line 3:
distWithoutRogue

Line 4:
constrainedDist

Sample Input 0

```
12 4
1 2 3 4
18
1 2 4 50
1 3 6 50
1 4 9 50
2 3 3 50
2 4 7 50
3 4 3 50
3 5 2 20
1 6 5 20
1 7 8 20
5 6 2 20
5 7 3 20
2 8 2 20
3 9 3 20
3 10 6 20
4 11 2 20
4 12 4 20
11 12 2 20
7 11 1 10
1 12 15
```

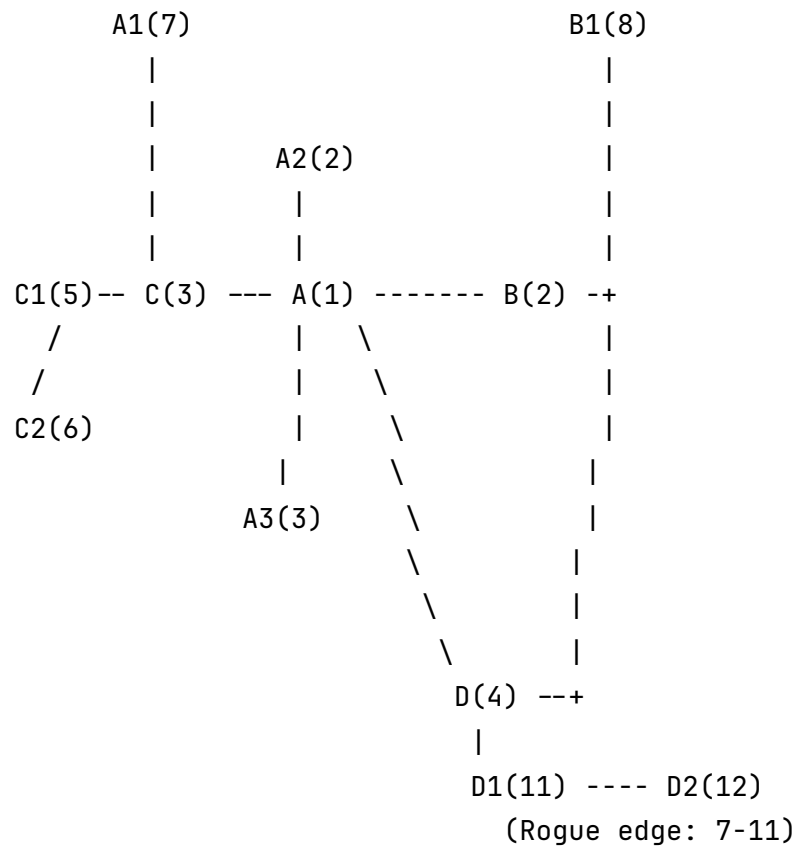
Sample Output 0

7 11
0 4 6 9 2 4 7 6 9 12 8 10
0 4 6 9 2 4 7 6 9 12 11 13
13

Explanation 0

Sample Input 0 — Network graph (numbers = edge latency)

Graph for Sample Input 0 – Network graph (numbers = edge latency)



Legend: Router (square), Device (circle), Rogue edge (dashed)

Node 7 and node 11 belong to different clusters — 7 is in router 1's cluster, 11 is in router 4's cluster — and edge 7-11 is the only edge in the graph that crosses clusters

without passing through a router. Removing it restores a valid mesh-and-clusters topology, so 7-11 is the rogue edge. With the rogue edge present, the shortest distance from node 1 to node 11 is 8 (path 1-5-6-7-11, cost $2+2+3+1$). Removing it, the shortest path becomes 1-4-11 (cost $9+2 = 11$). For the bandwidth query, edge 7-11 has bandwidth 10, below the required 15, so it cannot be used. The shortest path from node 1 to node 12 using only edges with bandwidth ≥ 15 is 1-4-12, with latency $9 + 4 = 13$.

Solution

Note: This is an advanced graph problem that combines topology validation, graph traversal, and shortest path algorithms. There is **no single standard algorithm** that directly solves it. The intended solution is a combination of several graph techniques.

Main Steps

Step 1 — Build the Graph

Store every edge as

```
(u, v, latency, bandwidth)
```

using an adjacency list.

Step 2 — Find the Rogue Edge

For every edge:

1. Temporarily remove it.
2. Check whether the remaining graph satisfies the required topology:

- Router nodes form a complete graph.
- Every non-router belongs to exactly one router's cluster.
- No cluster connects to another cluster.
- The graph remains connected.

Since the problem guarantees exactly one rogue edge, the first edge satisfying all conditions is the answer.

Step 3 — Run Dijkstra on the Original Graph

Using all edges, compute

```
distFull[]
```

from source s .

Step 4 — Remove the Rogue Edge

Delete the detected rogue edge. Run Dijkstra again. Store

```
distWithoutRogue[]
```

Step 5 — Bandwidth-Constrained Shortest Path

Build another graph containing only edges with

```
bandwidth  $\geq$  bMin
```

Run Dijkstra once more from `s`. If `t` is unreachable,

```
answer = -1
```

otherwise,

```
answer = dist[t]
```

Algorithms Used

Adjacency List

↓

Graph Validation

↓

Breadth-First Search / DFS

↓

Dijkstra (Original Graph)

↓

Dijkstra (Without Rogue Edge)

↓

Filtered Dijkstra

Complexity

Let

N = number of nodes

M = number of edges

Finding the rogue edge:

$$O(M \times (N + M))$$

(Each candidate edge may require validating the graph.) Each Dijkstra:

$$O((N + M) \log N)$$

Three Dijkstra runs:

$$O(3 \times (N + M) \log N)$$

Overall:

$$O(M(N + M) + (N + M) \log N)$$

This is acceptable for

$$N \leq 2000$$

$$M \leq 4000$$

Code :

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAXN 2005
#define MAXM 4005
#define INF 1000000000LL

typedef long long ll;

int n, r, m;
int routerLabel[MAXN]; /* routerLabel[i] = 1 if node i is a router */
int Eu[MAXM], Ev[MAXM], Ew[MAXM], Eb[MAXM];

/* ----- adjacency list (rebuilt for each graph variant) ----- */
int head_[MAXN];
int nxt_[2 * MAXM];
int to_[2 * MAXM];
int wt_[2 * MAXM];
int ecnt;

void addEdge(int u, int v, int w)
{
    to_[ecnt] = v;
    wt_[ecnt] = w;
    nxt_[ecnt] = head_[u];
    head_[u] = ecnt++;
}

/* excludeEdge = -1 means keep everything, otherwise skip that edge index */
/* bandMin = -1 means no bandwidth filter, otherwise keep only b ≥ bandMin */
void buildGraph(int excludeEdge, int bandMin)
{
    for (int i = 1; i ≤ n; i++)
        head_[i] = -1;
    ecnt = 0;

    for (int i = 0; i < m; i++)

```

```

{
    if (i == excludeEdge)
        continue;
    if (bandMin != -1 && Eb[i] < bandMin)
        continue;

    addEdge(Eu[i], Ev[i], Ew[i]);
    addEdge(Ev[i], Eu[i], Ew[i]);
}
}

/* ----- simple O(N^2) Dijkstra (fine for N ≤ 2000) ----- */
ll dist_[MAXN];
int visited_[MAXN];

void dijkstra(int src)
{
    for (int i = 1; i ≤ n; i++)
    {
        dist_[i] = INF;
        visited_[i] = 0;
    }
    dist_[src] = 0;

    for (int iter = 1; iter ≤ n; iter++)
    {
        int u = -1;
        ll best = INF + 1;

        for (int i = 1; i ≤ n; i++)
        {
            if (!visited_[i] && dist_[i] < best)
            {
                best = dist_[i];
                u = i;
            }
        }

        if (u == -1)
            break;
    }
}

```

```

    visited_[u] = 1;

    for (int e = head_[u]; e != -1; e = nxt_[e])
    {
        int v = to_[e];
        ll nd = dist_[u] + wt_[e];

        if (nd < dist_[v])
            dist_[v] = nd;
    }
}

/* ----- Union-Find used to validate cluster structure ----- */
int parent_[MAXN];

int find(int x)
{
    if (parent_[x] == x)
        return x;

    parent_[x] = find(parent_[x]);
    return parent_[x];
}

void unite(int a, int b)
{
    a = find(a);
    b = find(b);

    if (a != b)
        parent_[a] = b;
}

int routerCountInComponent[MAXN];

/*
 * Checks whether excluding edge `excludeEdge` restores a valid
 * mesh-and-clusters topology.

```

```

*
* A router-router edge can never be the rogue edge: routers are already
* guaranteed to form a complete mesh and the graph has no multi-edges,
* so removing any router-router edge instantly breaks the mesh. Such
* candidates are rejected in  $O(1)$  without scanning every router pair.
*
* For every other candidate, we union every remaining non-router-router
* edge (device-device and router-device edges). In a valid topology,
* every resulting component must contain EXACTLY one router – this is
* "every device belongs to exactly one cluster" + "graph stays
* connected" from the problem statement, checked together in one
*  $O(N + M)$  pass.
*/
int isValidWithoutEdge(int excludeEdge)
{
    if (routerLabel[Eu[excludeEdge]] && routerLabel[Ev[excludeEdge]])
        return 0; /* router-router edge → mesh would break */

    for (int i = 1; i ≤ n; i++)
        parent_[i] = i;

    for (int i = 0; i < m; i++)
    {
        if (i == excludeEdge)
            continue;

        if (routerLabel[Eu[i]] && routerLabel[Ev[i]])
            continue; /* mesh edges don't define cluster membership */

        unite(Eu[i], Ev[i]);
    }

    for (int i = 1; i ≤ n; i++)
        routerCountInComponent[i] = 0;

    for (int i = 1; i ≤ n; i++)
        if (routerLabel[i])
            routerCountInComponent[find(i)]++;

    for (int i = 1; i ≤ n; i++)

```

```

        if (routerCountInComponent[find(i)]  $\neq$  1)
            return 0;

    return 1;
}

int main()
{
    scanf("%d %d", &n, &r);

    int routers[MAXN];
    for (int i = 0; i < r; i++)
    {
        scanf("%d", &routers[i]);
        routerLabel[routers[i]] = 1;
    }

    scanf("%d", &m);

    for (int i = 0; i < m; i++)
        scanf("%d %d %d %d", &Eu[i], &Ev[i], &Ew[i], &Eb[i]);

    int s, t, bMin;
    scanf("%d %d %d", &s, &t, &bMin);

    /* ----- Step 2: find the rogue edge ----- */
    int rogue = -1;
    for (int i = 0; i < m; i++)
    {
        if (isValidWithoutEdge(i))
        {
            rogue = i;
            break;
        }
    }

    int ru = Eu[rogue], rv = Ev[rogue];
    if (ru > rv)
    {
        int tmp = ru;

```

```

    ru = rv;
    rv = tmp;
}

/* ----- Step 3: Dijkstra on the full graph ----- */
buildGraph(-1, -1);
dijkstra(s);

ll distFull[MAXN];
for (int i = 1; i ≤ n; i++)
    distFull[i] = dist_[i];

/* ----- Step 4: Dijkstra with the rogue edge removed ----- */
buildGraph(rogue, -1);
dijkstra(s);

ll distWithoutRogue[MAXN];
for (int i = 1; i ≤ n; i++)
    distWithoutRogue[i] = dist_[i];

/* ----- Step 5: bandwidth-constrained shortest path ----- */
buildGraph(-1, bMin);
dijkstra(s);

ll constrainedDist = (dist_[t] ≥ INF) ? -1 : dist_[t];

/* ----- Output ----- */
printf("%d %d\n", ru, rv);

for (int i = 1; i ≤ n; i++)
    printf("%lld%c", distFull[i], i == n ? '\n' : ' ');

for (int i = 1; i ≤ n; i++)
    printf("%lld%c", distWithoutRogue[i], i == n ? '\n' : ' ');

printf("%lld\n", constrainedDist);

return 0;
}

```


Thinking Process

1. What is the Problem Asking?

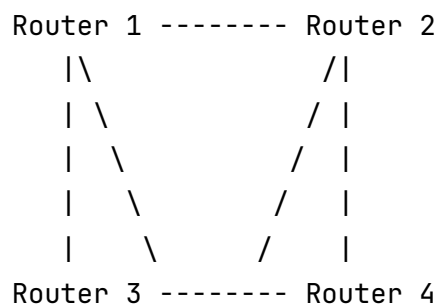
At first glance, the problem seems overwhelming because it asks us to perform **three different tasks**:

1. Find the rogue edge.
2. Compute shortest paths.
3. Compute a bandwidth-constrained shortest path.

Instead of solving everything at once, break the problem into independent parts.

2. Understanding the Network Structure

The network should look like this:



Every router is connected to every other router. Each router also owns its own local cluster. Example:

Router 1

|

A1

/ \

A2 A3

Router 2

|

B1

Router 3

|

C1

Devices inside a cluster may connect however they like, but they can never connect to another cluster.

3. What Went Wrong?

Exactly one edge violates this rule. Example:

A2 ----- B1

This edge connects

Cluster A

↓

Cluster B

which should never happen. This is the rogue edge.

4. How Can We Find It?

The problem guarantees:

```
Removing exactly one edge  
restores the correct topology.
```

This is a huge hint. Instead of trying to directly detect the bad edge, try every edge. For each edge:

```
Remove it
```

↓

```
Check whether the graph  
becomes a valid network.
```

The only edge that works is the rogue edge.

5. Shortest Paths

Once the rogue edge is known, the remaining tasks become straightforward. For the original graph:

```
Run Dijkstra.
```

Store

```
distFull[]
```

For the repaired graph:

Remove the rogue edge

↓

Run Dijkstra again.

Store

```
distWithoutRogue[]
```

6. Bandwidth Constraint

Now another condition appears. Only edges satisfying

$$\text{bandwidth} \geq \text{bMin}$$

may be used. Instead of modifying Dijkstra, simply ignore edges whose bandwidth is too small. Run Dijkstra on this filtered graph. If the target cannot be reached,

Answer = -1

otherwise,

Answer = shortest distance.

7. Final Strategy

The complete workflow is

Read Graph

↓

Try removing every edge

↓

Validate Topology

↓

Found Rogue Edge

↓

Dijkstra (Original Graph)

↓

Remove Rogue Edge

↓

Dijkstra Again

↓

Filter by Bandwidth

↓

Dijkstra Again

Notice that this problem is **not** about inventing a new algorithm. Instead, it combines several well-known graph techniques into one solution.

Dry Run

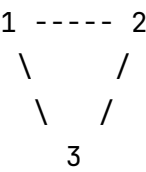
Sample Graph

Suppose the network looks like this:

Routers:

- 1
- 2
- 3

Since routers form a complete mesh,



Cluster of Router 1

- 1
- |
- 4
- |
- 5

Cluster of Router 2

2
|
6

Cluster of Router 3

3
|
7

Now suppose there is one illegal edge

5 ----- 6

This connects

Cluster 1

↓

Cluster 2

which violates the topology.

Step 1 — Find the Rogue Edge

Try removing each edge one by one.

Remove

1 – 2

Router mesh breaks. ❌ Not valid.

Remove

4 – 5

Cluster disconnects. ❌ Not valid.

Remove

5 – 6

Now

- Router mesh is complete.
- Every device belongs to exactly one router.
- No cluster connects to another cluster.
- Graph satisfies all rules.

Therefore,

Rogue Edge

=

(5,6)

Step 2 — Dijkstra on Full Graph

Suppose

Start = 1

Shortest distances become

Node

1 → 0

2 → 4

3 → 3

4 → 2

5 → 5

6 → 6

7 → 4

Store as

```
distFull[]
```

Step 3 — Remove Rogue Edge

Delete

5 – 6

Run Dijkstra again. Some shortest paths become longer because the shortcut has disappeared. Example

Node

6

Old Distance = 6

New Distance = 9

Store these values in

```
distWithoutRogue[]
```

Step 4 — Bandwidth Constraint

Suppose

$b_{\min} = 20$

Only keep edges satisfying

$\text{bandwidth} \geq 20$

Filtered graph

1 ----- 2

|

3

|

7

Edges with smaller bandwidth are ignored. Run Dijkstra again. Suppose

Shortest Path

$1 \rightarrow 3 \rightarrow 7$

Distance = 8

Then

constrainedDist = 8

If no valid path exists,

```
constrainedDist = -1
```

Final Output

```
Rogue Edge
```

```
(5,6)
```

```
distFull[]
```

```
0 4 3 2 5 6 4
```

```
distWithoutRogue[]
```

```
0 4 3 2 5 9 4
```

```
constrainedDist
```

```
8
```

This demonstrates the complete workflow:

Find Rogue Edge

↓

Run Dijkstra

↓

Remove Rogue Edge

↓

Run Dijkstra Again

↓

Apply Bandwidth Filter

↓

Run Dijkstra Again

Complexity Analysis

Let

N = Number of Nodes

M = Number of Edges

1. Finding the Rogue Edge

There are

M

edges. In the worst case, we try removing every edge once. For each candidate edge, we verify whether the remaining graph satisfies the required topology. A graph validation using DFS/BFS takes

$O(N + M)$

Therefore,

Time

=

$O(M \times (N + M))$

2. Dijkstra on the Original Graph

Using

- Adjacency List
- Min Priority Queue

the complexity is

$O((N + M) \log N)$

3. Dijkstra After Removing the Rogue Edge

Exactly the same complexity:

$$O((N + M) \log N)$$

4. Bandwidth-Constrained Dijkstra

Ignore all edges whose

$$\text{bandwidth} < b_{\text{Min}}$$

The remaining graph still has at most

$$M$$

edges. Therefore,

$$O((N + M) \log N)$$

Overall Time Complexity

Combining everything,

Finding Rogue Edge

$$O(M(N + M))$$

plus

Three Dijkstra Runs

$$3 \times O((N + M) \log N)$$

Overall,

$$O(M(N + M) + (N + M) \log N)$$

Since

$$N \leq 2000$$

$$M \leq 4000$$

this easily fits within typical competitive programming limits.

Space Complexity

The graph is stored as an adjacency list. Memory required: Adjacency List

$$O(N + M)$$

Distance Arrays

distFull

distWithoutRogue

$O(N)$

Visited Array

$O(N)$

Priority Queue Worst case:

$O(M)$

Total Space Complexity:

$O(N + M)$

Final Complexity Summary

Operation	Complexity
Rogue Edge Detection	$O(M(N + M))$
Dijkstra (Full Graph)	$O((N + M) \log N)$
Dijkstra (Without Rogue Edge)	$O((N + M) \log N)$
Bandwidth-Constrained Dijkstra	$O((N + M) \log N)$

Operation	Complexity
Overall Time	$O(M(N + M) + (N + M) \log N)$
Space	$O(N + M)$

Common Mistakes

1. Assuming the Rogue Edge Has the Largest Weight

Some programmers try to identify the rogue edge based on its latency or bandwidth. ❌
 Wrong The rogue edge is **not** necessarily the edge with the largest latency or the smallest bandwidth. It is identified **only** by the network topology.

2. Forgetting the Router Mesh Property

The router nodes must form a **complete graph**. That means every pair of routers must be directly connected. While validating a candidate graph, always verify that this property still holds.

3. Ignoring Cluster Boundaries

A common mistake is checking only graph connectivity. A valid network must also satisfy:

Every non-router belongs to exactly one router's cluster.

No device may belong to multiple clusters.

4. Using BFS Instead of Dijkstra

Edge latencies are weighted. Therefore, ✗

BFS

does **not** compute shortest paths. Always use

Dijkstra's Algorithm

for all shortest-path computations.

5. Forgetting to Remove the Rogue Edge

The problem asks for two different distance arrays. Some solutions compute

`distFull`

correctly, but forget to actually remove the rogue edge before computing

`distWithoutRogue`.

This produces identical outputs, which is incorrect.

6. Ignoring the Bandwidth Constraint

For the constrained query, only edges satisfying

$$\text{bandwidth} \geq \text{bMin}$$

may be used. Do **not** filter edges by latency. The filter applies only to the bandwidth field.

7. Not Handling Unreachable Nodes

After removing the rogue edge or applying the bandwidth filter, some nodes may become unreachable. Return

-1

for unreachable destinations as required by the problem.

8. Modifying the Original Graph Permanently

While testing candidate rogue edges, avoid permanently deleting edges. Either:

- temporarily ignore the edge during traversal, or
- restore it after validation.

Otherwise, later iterations will run on an incorrect graph.

9. Running Dijkstra Too Many Times

After identifying the rogue edge, only three Dijkstra runs are needed:

1. Full graph
2. Graph without the rogue edge
3. Bandwidth-filtered graph

Running Dijkstra repeatedly during validation is unnecessary and significantly slows the solution.

Pattern Recognition

What Pattern Does This Problem Follow?

This is a **multi-concept graph problem**. Unlike standard contest problems, it cannot be solved using a single algorithm. Instead, it combines several graph techniques.

Key Observation

The statement guarantees:

```
Removing exactly one edge  
restores the correct topology.
```

This immediately suggests the following approach:

Try removing every edge

↓

Validate the graph

↓

The only valid one
is the rogue edge.

Algorithms Used

1. Graph Representation

Use an

Adjacency List

to efficiently store the graph. Time:

$O(N + M)$

2. Graph Validation

After removing a candidate edge, verify:

- Router mesh is complete.
- Every device belongs to exactly one router.

- No cross-cluster connections exist.
- Graph remains connected.

This validation is naturally performed using

DFS

or

BFS.

3. Dijkstra's Algorithm

Since every edge has a

Latency (Weight)

the shortest paths must be computed using

Dijkstra.

It is executed three times:

Original Graph

↓

Graph Without Rogue Edge

↓

Bandwidth-Filtered Graph

4. Edge Filtering

For the constrained query, ignore every edge whose

$\text{Bandwidth} < b_{\text{Min}}$.

Then simply run Dijkstra again. No new shortest-path algorithm is required.

Overall Pattern

Weighted Graph



Try Removing Every Edge



Graph Validation



Find Rogue Edge



Dijkstra



Remove Rogue Edge



Dijkstra



Filter Edges by Bandwidth



Dijkstra

Similar Competitive Programming Problems

If a problem contains phrases like

- invalid edge
- corrupted graph
- faulty connection
- remove one edge
- restore topology
- shortest path after modification
- constrained shortest path

then think about combining

Graph Validation

+

DFS / BFS

+

Dijkstra

instead of searching for a single specialized algorithm.

Main Pattern Learned

Topology Verification

+

Shortest Path

+

Edge Filtering

+

Simulation

This problem is an excellent example of a **hybrid graph problem**, where multiple standard algorithms work together to solve different parts of the task.

Summary

Core Idea

This problem combines **network topology verification** with **shortest path algorithms**. The graph is almost a valid **mesh-and-clusters** network, except for **one rogue edge** connecting two different clusters. Our goal is to:

1. Identify the rogue edge.
2. Compute shortest paths before removing it.
3. Compute shortest paths after removing it.
4. Compute a bandwidth-constrained shortest path.

Complete Approach

Step 1

Build the graph using an adjacency list.

Input

↓

Adjacency List

Step 2

Find the rogue edge. For every edge:

Remove Edge

↓

Validate Graph

The edge whose removal restores the required topology is the answer.

Step 3

Run Dijkstra on the original graph. Store

`distFull[]`

Step 4

Remove the rogue edge. Run Dijkstra again. Store

```
distWithoutRogue[]
```

Step 5

Apply the bandwidth restriction. Ignore every edge satisfying

```
bandwidth < bMin
```

Run Dijkstra once more. If the destination is unreachable,

```
Answer = -1
```

otherwise,

```
Answer = shortest latency
```

Algorithms Used

Adjacency List

↓

DFS / BFS (Topology Validation)

↓

Dijkstra

↓

Edge Filtering

↓

Dijkstra

Time Complexity

Finding the rogue edge

$O(M(N + M))$

Three Dijkstra runs

$3 \times O((N + M) \log N)$

Overall

$$O(M(N + M) + (N + M) \log N)$$

Space Complexity

$$O(N + M)$$

Key Concepts Learned

- Graph Representation
- Network Topology Validation
- DFS / BFS Traversal
- Dijkstra's Shortest Path
- Edge Filtering
- Simulation-Based Problem Solving

Pattern

Weighted Graph

↓

Try Removing Every Edge

↓

Validate Topology

↓

Find Rogue Edge

↓

Run Dijkstra

↓

Repair Graph

↓

Run Dijkstra

↓

Apply Constraints

↓

Run Dijkstra

Key Takeaway

The most important observation is:

The problem is NOT asking for
a new graph algorithm.

Instead, it asks you to **combine multiple standard graph techniques** into a single solution. The complete toolkit is:

Graph Representation

+

DFS / BFS

+

Topology Validation

+

Dijkstra

+

Edge Filtering

Many advanced competitive programming problems follow this style—where recognizing **how to combine known algorithms** is more important than inventing a new one.